



信息与软件工程学院

编译技术

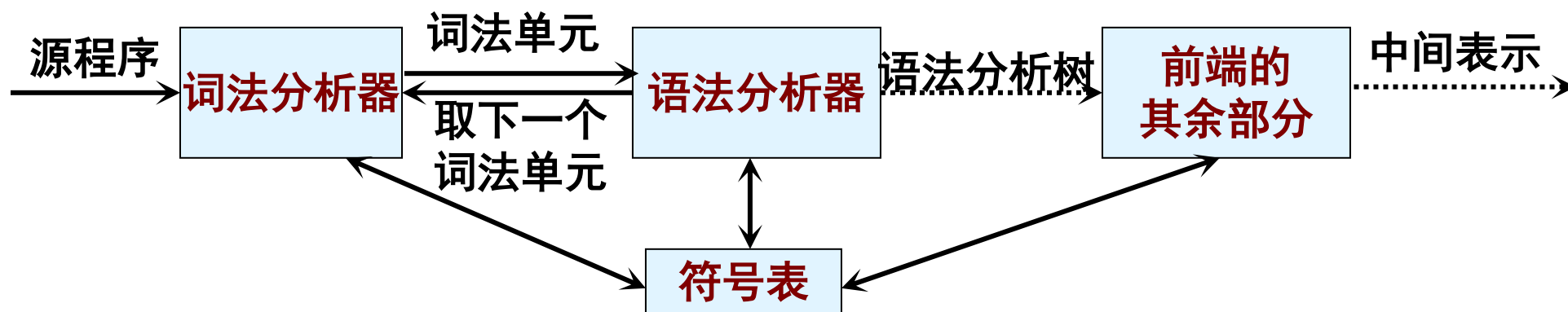
主讲教师： 陈安龙

第3章 语法分析

- 语法分析简介
- 上下文无关文法
- 自上而下递归下降分析
- 自下而上分析法
- LR(0),SLR(1),LR(1),LALR(1)分析法

1 · 语法分析简介

- 语法分析（Syntax Analysis），也称为解析器（Parser），是编译过程中的第二个阶段，紧随词法分析之后。语法分析的主要任务是**根据源代码中识别出的词法单元（Token）序列**，按照给定的**语法规则**（通常是上下文无关文法）来构建**抽象语法树（Abstract Syntax Tree, AST）**或**语法分析树（Parse Tree）**，以便后续的语义分析、中间代码生成等阶段使用。



语法分析的主要任务包括：

- ① **确定语法结构**：根据语法规则，确定源代码中各个词法单元之间的语法结构，即如何组合成合法的语句或表达式；并将其转换为易于处理的数据结构，为后续的处理阶段提供必要的输入。
- ② **错误检测**：识别并报告源代码中的语法错误，如缺少分号、括号不匹配等。
- ③ **构建语法树**：根据语法规则，将识别出的词法单元组织成树状结构，以表示源代码的语法结构。

语法分析的方法：

- ① 语法分析器通常根据给定的语法规则（通常是上下文无关文法）来进行分析。
常见的语法分析算法包括自顶向下分析（Top-Down Parsing）和自底向上分析（Bottom-Up Parsing）。
- ② 语法分析的输出通常是抽象语法树（AST），它是源代码的树状表示形式，其中每个节点表示一个语法结构，如表达式、语句等，而子节点表示该结构的组成部分。

示例：解析带有条件语句的代码片段

- 假设有如下简单的C语言代码片段：

```
if (x > 0) {  
    y = x * 2;  
} else {  
    y = -x;  
}
```

(1) 词法分析阶段

词法分析器将代码拆分成记号（tokens），例如：

Token类型	Lexeme	Token类型	Lexeme
关键字 (IF)	if	常数 (NUM)	2
分隔符 (LPAREN)	(	分隔符 (SEMICOLON)	;
标识符 (ID)	x	分隔符 (RBRACE)	}
运算符 (GT)	>	关键字 (ELSE)	else
常数 (NUM)	0	分隔符 (LBRACE)	{
分隔符 (RPAREN)	)	标识符 (ID)	y
分隔符 (LBRACE)	{	运算符 (ASSIGN)	=
标识符 (ID)	y	运算符 (MINUS)	-
运算符 (ASSIGN)	=	标识符 (ID)	x
标识符 (ID)	x	分隔符 (SEMICOLON)	;
运算符 (MULT)	*	分隔符 (RBRACE)	}

(2) 语法分析阶段

语法分析器根据语言的语法规则，判断这些记号是否构成合法的语句，并构建语法树。

语法规则示例（简化版）：

语句 $\rightarrow$ if (条件) 语句块 else 语句块

条件 $\rightarrow$ 表达式 比较运算符 表达式

语句块 $\rightarrow$ { 语句列表 }

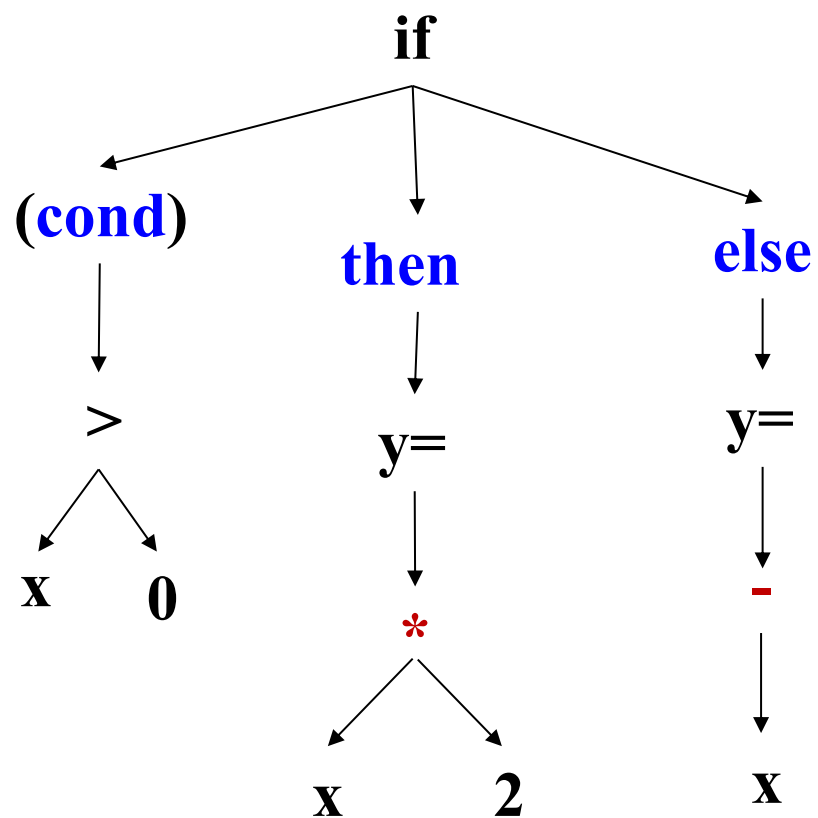
语句列表 $\rightarrow$ 语句 语句列表 $\mid \varepsilon$

语句 $\rightarrow$ 赋值语句 $\mid$ 其他语句

赋值语句 $\rightarrow$ 标识符 = 表达式 ;

表达式 $\rightarrow$ 标识符 $\mid$ 数字 $\mid$ 表达式 运算符 表达式 $\mid$ 一元运算符 表达式

生成的语法树（部分示意）



条件部分 $x > 0$ 是一个比较表达式

if 语句结构完整，有对应的 else 分支

两个语句块分别包含赋值语句

赋值语句中表达式结构正确，如 $x * 2$ 和 $-x$

(3) 语法分析的意义

- ① 语法分析帮助编译器理解程序的结构和层次，明确“条件判断”、“代码块”、“赋值操作”等关系。
- ② 如果代码写成 `if x > 0 y = x * 2; else y = -x;`（缺少括号），语法分析器会报错，提示语法不正确。语法树为后续的语义分析和代码生成提供了基础。
- ③ 语法分析就是把代码的“单词”按照语言的语法规则组合成“句子”和“段落”，形成一棵树状结构，帮助编译器理解程序的逻辑和结构

如何表示语言的语法结构？

- 在编译技术中，常用的方法是使用上下文无关文法（Context-Free Grammar, CFG）来表示某种语言的语法结构。
- 上下文无关文法由一组产生式组成，描述了语言中各个句子的结构。
- 产生式是一种形式化的规则，用于描述上下文无关文法（CFG）中的语法结构。它由一个非终结符（左部）和一个由终结符和/或非终结符组成的符号串（右部）组成，表示了如何将一个非终结符替换为一个符号串。
- 形式化地，产生式的形式可以表示为 $A \rightarrow \alpha$ ，其中 A 是一个非终结符， α 是一个由终结符和/或非终结符组成的符号串。

上下文无关文法的形式化描述

- 上下文无关文法是四元组 (V_T, V_N, S, P)

V_T : 终结符集合

V_N : 非终结符集合

S : 开始符号, 非终结符中的一个

P : 产生式集合, 产生式形式: $A \rightarrow \alpha$, $A \in V_N$, $\alpha \in V_N \cup V_T$

表达式文法示例

终结符: $+, -, *, /, \text{id}, (,)$

非终结符: $\text{expression}, \text{term}, \text{factor}$

定义产生式:

$\text{expression} \rightarrow \text{expression} + \text{term} \mid \text{expression} - \text{term} \mid \text{term}$

$\text{term} \rightarrow \text{term} * \text{factor} \mid \text{term} / \text{factor} \mid \text{factor}$

$\text{factor} \rightarrow (\text{expression}) \mid \text{id}$

id 表示非终结符数字;

上述产生文法可以用下列简约表示

- $E \rightarrow E + T \mid E - T \mid T$
- $T \rightarrow T * F \mid T / F \mid F$
- $F \rightarrow (E) \mid id$

用产生式描述if条件语句的语法结构

假设C语言的if-else 语句有如下结构形式：

if (表达式) { 语句 } **else** { 语句 }

我们如何来构造规则来表达C语言的if-else语法？

使用变量 **expr**表示表达式； **stmt**表示语句，则if-else语法的可以由如下规则描述：

$\text{stmt} \rightarrow \text{if}(\text{expr}) \{ \text{stmt} \} \text{else} \{ \text{stmt} \}$

if、**else**、圆括号和花括号等称为终结符；**expr**、**stmt**为非终结符。

文法产生式的推导

- 产生式的推导是指根据文法的产生式规则，从起始符号（通常是文法的开始符号）出发，通过一系列推导步骤生成一个或多个句子的过程。产生式的推导过程可以帮助我们理解文法规则是如何被应用以生成语言中的句子。步骤如下：
 - ① 初始状态：从文法的起始符号开始，作为推导的初始状态。
 - ② 选择产生式：根据当前推导所处的状态（即当前推导所得的符号串），选择一个适当的产生式进行推导。通常有多个产生式可以选择，需要根据文法规则进行选择。
 - ③ 替换：根据选择的产生式，将符号串中非终结符用产生式右部（终结符或非终结符序列）替换，从而生成新的符号串。
 - ④ 重复：重复步骤2和步骤3，直到无法再应用产生式进行推导。

产生式的推导过程示例

- 例：文法 $E \rightarrow E + E \mid E * E \mid (E) \mid -E \mid \text{id}$

$$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E + E) \Rightarrow -(\text{id} + E) \Rightarrow -(\text{id} + \text{id})$$

- 记号：

$S \Rightarrow^* \alpha$ 表示0步或多步推导得到句子 α ,

$S \Rightarrow^+ \alpha$ 表示1步或多步推导得到句子 α 。

最左推导

- 在文法的推导过程中,每一步推导总是选择最左边的非终结符进行替换,直到推导出目标符号串,称为最左推导。

- 例: 文法 $E \rightarrow E + E \mid E * E \mid (E) \mid -E \mid \text{id}$

$$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E + E) \Rightarrow -(\text{id} + E) \Rightarrow -(\text{id} + \text{id})$$

- 也可用下列记号表示最左推导:

$$\begin{aligned} E &\Rightarrow_{lm} -E \\ &\Rightarrow_{lm} -(E) \\ &\Rightarrow_{lm} -(E + E) \\ &\Rightarrow_{lm} -(\text{id} + E) \\ &\Rightarrow_{lm} -(\text{id} + \text{id}) \end{aligned}$$

最右推导

- 与最左推导相反，在推导过程中，**每一步推导总是选择最右边的非终结符进行替换**，直到推导出目标符号串，**称为最右推导，又称为规范推导**。

- 例：文法 $E \rightarrow E + E \mid E * E \mid (E) \mid -E \mid \text{id}$

$$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E + E) \Rightarrow -(E + \text{id}) \Rightarrow -(\text{id} + \text{id})$$

- 也可用下列记号表示最右推导：

$$\begin{aligned} E &\Rightarrow_{rm} -E \\ &\Rightarrow_{rm} -(E) \\ &\Rightarrow_{rm} -(E + E) \\ &\Rightarrow_{rm} -(E + \text{id}) \\ &\Rightarrow_{rm} -(\text{id} + \text{id}) \end{aligned}$$

与推导相关的概念

(1) **句型**：是由终结符和非终结符组成的符号串，可以通过文法的推导规则生成句子。句型是语法分析中用来描述句子结构的概念。

S 是文法 G 的开始符号，如果**推导** $S \xRightarrow{*} \alpha\beta\delta$ ，则称 $\alpha\beta\delta$ 是文法 G 的**句型**。

(2) **短语**：若 $S \xRightarrow{*} \alpha A \delta$ 且 $A \xRightarrow{+} \beta$ ，则称 β 是**句型** $\alpha\beta\delta$ 的**短语**。

(3) **直接短语**：若 $S \xRightarrow{*} \alpha A \delta$ 且 $A \Rightarrow \beta$ ，则称 β 是**句型** $\alpha\beta\delta$ 的**直接短语**。

(4) **句柄**：某个句型的**最左直接短语**称为该句型的**句柄**。

示例:

对文法:

$E \rightarrow E+T | E-T | T$

$T \rightarrow T * F | T / F | F$

$F \rightarrow i | (E)$

求句型 $E+T * F+i$ 的短语、直接短语和句柄。

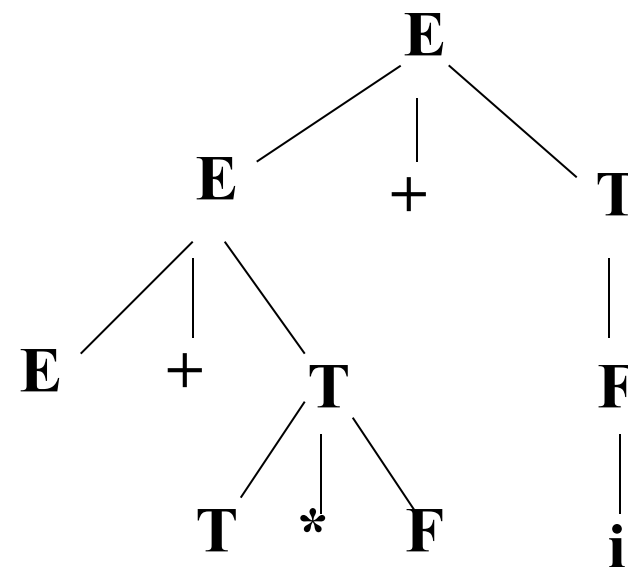
推导出该句型的过程为:

$E \Rightarrow E+T \Rightarrow E+F \Rightarrow E+i \Rightarrow E+T+i \Rightarrow E+T * F+i$ 是规范句型。

短语: $E+T * F+i$, $E+T * F$, $T * F$, i

直接短语: $T * F$, i

句柄: $T * F$



归约

- 在语法分析中，规约通常是在符号串中匹配到一个产生式右部时，将其替换为该产生式左部的非终结符。
- 如果归约是最右推导的逆过程，则称为规范规约。

规范规约的示例：

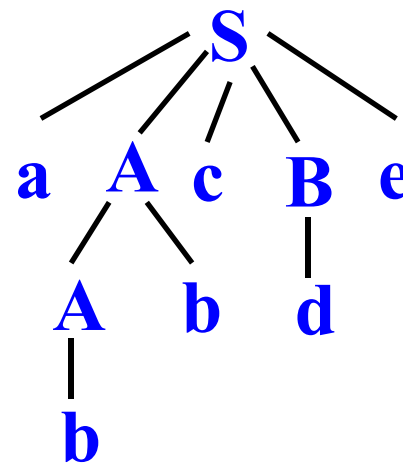
如文法：

$S \rightarrow aAcBe$

$A \rightarrow Ab|b$

$B \rightarrow d$

分析输入串： abbcde



$S \Rightarrow aAcBe \Rightarrow aAcde \Rightarrow aAbcde \Rightarrow abbcde$

$S \Leftarrow S \rightarrow aAcBe \Leftarrow B \rightarrow d \Leftarrow A \rightarrow Ab \Leftarrow A \rightarrow b$

每次归约的都是句型中的句柄。

活前缀（也称为可行前缀）

- 推导过程: $S \xRightarrow{*} \delta A \gamma \Rightarrow \delta \alpha \beta \gamma$, $A \rightarrow \alpha \beta$ 是一次最右推(规范推导), 对于句型 $\delta \alpha \beta \gamma$, $\alpha \beta$ 是一个句柄。如果 $\delta \alpha \beta = u_1 u_2 \dots u_n$, 则符号串 $u_1 u_2 \dots u_i$ ($1 \leq i \leq n$) 是句型 $\delta \alpha \beta \gamma$ 的活前缀。

例: 文法 $E \rightarrow E+T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow (E) \mid \text{id}$

规范句型 $T * \text{id} + \text{id}$ 的最右推导: $E \Rightarrow E+T \Rightarrow E+\text{id} \Rightarrow T+\text{id} \Rightarrow T * F + \text{id} \Rightarrow T * \text{id} + \text{id}$

这里用到产生式 $F \rightarrow \text{id}$ 推导, 对应前面的 $\delta \alpha \beta = T * \text{id}$

则: $T * \text{id} + \text{id}$ 的活前缀是: $\varepsilon, T, T*, T * \text{id}$

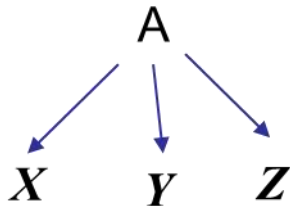
活前缀与句柄存在下列某种关系

- ① 活前缀已含有句柄的全部符号 (可归约) $A \rightarrow \alpha\beta\cdot$, 产生式右部 $\alpha\beta$ (句柄) 已经出现在栈顶, 用 $A \rightarrow \alpha\beta$ 进行归约;
- ② 活前缀只含有句柄的一部分符号, $A \rightarrow \alpha\cdot\beta$, α 已经出现在栈顶, 期待从余留输入串中看到能由 β 推出的符号串。
- ③ 活前缀不含有句柄的任何符号, $A \rightarrow \cdot\alpha\beta$, 期待从余留输入串中能看到 $\alpha\beta$ 推出的串。

语法分析树

- 语法分析树是以图形方式展现从文法开始符号推导出语言中字符串的过程。
- 给定的上下无关文法，该文法的一棵语法分析树具有如下性质：
 - ① 根节点的标号为文法的开始符号
 - ② 每个叶子结点的标号为终结符号或 ε
 - ③ 每个内部结点的标号为非终结符号
 - ④ 如果非终结符号A是某个内部结点的标号，并且它的子节点的标号从左到右分别为 $X_1, X_2, \dots, X_n$ ，那么必有产生式 $A \rightarrow X_1 X_2 \dots X_n$ ； $X_1, X_2, \dots, X_n$ 可以为终结符，也可以为非终结符；如果有产生式 $A \rightarrow \varepsilon$ ，则A只有一个标号为 ε 的子结点。

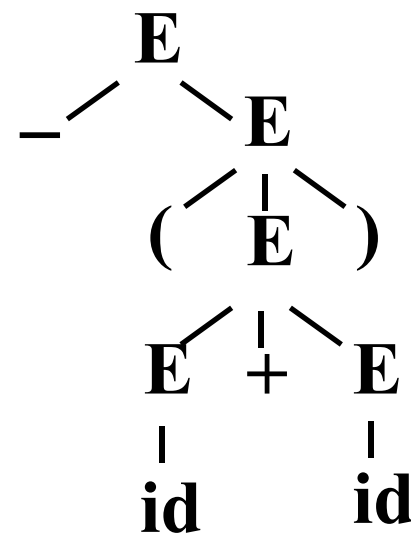
⑤ 如： $A \rightarrow XYZ$



语法分析树

- 例：文法 $E \rightarrow E + E \mid E * E \mid (E) \mid -E \mid \text{id}$

$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E + E) \Rightarrow -(\text{id} + E) \Rightarrow -(\text{id} + \text{id})$



文法的二义性

- 对于某个文法，如果有多棵语法分析树生成同一个终结字符串，则称文法具有二义性

$$E \Rightarrow E * E$$

$$\Rightarrow \text{id} * E$$

$$\Rightarrow \text{id} * E + E$$

$$\Rightarrow \text{id} * \text{id} + E$$

$$\Rightarrow \text{id} * \text{id} + \text{id}$$

$$E \Rightarrow E + E$$

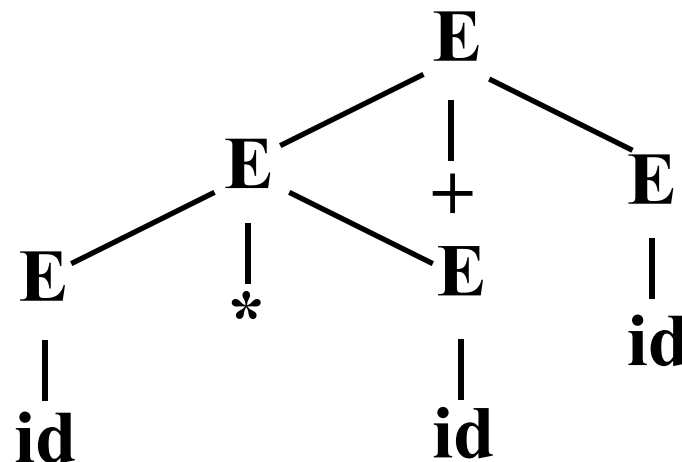
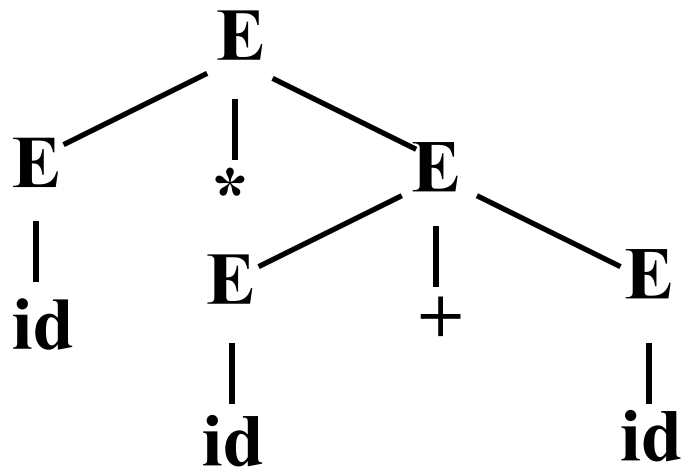
$$\Rightarrow E * E + E$$

$$\Rightarrow \text{id} * E + E$$

$$\Rightarrow \text{id} * \text{id} + E$$

$$\Rightarrow \text{id} * \text{id} + \text{id}$$

两棵不同的语法树



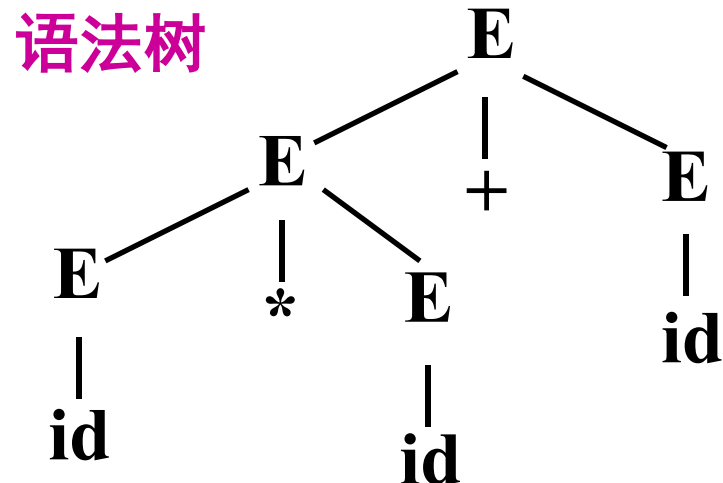
如何消除文法的二义性

- 如果使用二义性文法构造的编译器，对于相同的程序代码，由于可能构造多棵不同的语法分析树，最终导致源程序代码编译成目标代码的不确定性。
- 可以使用附件规则来消除文法的二义性，如：
 - ① 运算符的结合性
 - ② 运算符的优先级
 - ③ 修改文法，构造无二义性的文法

利用优先级来消除文法二义性

- 对于+ - * /四则算术运算，可以利用优先级来消除二义性
- 例：文法 $E \rightarrow E + E \mid E * E \mid (E) \mid -E \mid \text{id}$
- 假设要推导出 $\text{id} * \text{id} + \text{id}$

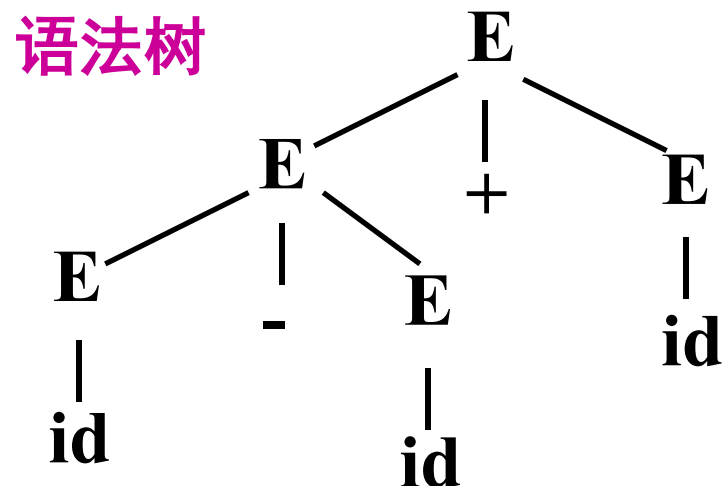
$E \Rightarrow E + E$
 $\Rightarrow E * E + E$
 $\Rightarrow \text{id} * E + E$
 $\Rightarrow \text{id} * \text{id} + E$
 $\Rightarrow \text{id} * \text{id} + \text{id}$



利用结合性来消除文法二义性

- 对于+ - * /四则算术运算，可以利用做左结合性来消除二义性
- 例：文法 $E \rightarrow E + E \mid E - E \mid (E) \mid -E \mid \text{id}$
- 假设要推导出id-id+id

$E \Rightarrow E + E$
 $\Rightarrow E - E + E$
 $\Rightarrow \text{id} - E + E$
 $\Rightarrow \text{id} - \text{id} + E$
 $\Rightarrow \text{id} - \text{id} + \text{id}$



改写文法消除文法二义性

下列文法是具有二义性

$stmt \rightarrow \text{if } expr \text{ then } stmt$
 $| \text{if } expr \text{ then } stmt \text{ else } stmt$
 $| \text{other}$

- 句型: $\text{if } expr \text{ then if } expr \text{ then } stmt \text{ else } stmt$
- 两个最左推导:

$stmt \Rightarrow \text{if } expr \text{ then } stmt \Rightarrow \text{if } expr \text{ then if } expr \text{ then } stmt \text{ else } stmt$

$stmt \Rightarrow \text{if } expr \text{ then } stmt \text{ else } stmt \Rightarrow \text{if } expr \text{ then if } expr \text{ then } stmt \text{ else } stmt$

将上述文法修改为无二义性文法

■ 无二义的文法

$stmt \rightarrow matched_stmt \mid unmatched_stmt$

$matched_stmt \rightarrow \text{if } expr \text{ then } matched_stmt \text{ else } matched_stmt \mid \text{other}$

$unmatched_stmt \rightarrow \text{if } expr \text{ then } stmt \mid \text{if } expr \text{ then } matched_stmt \text{ else } unmatched_stmt$

左递归文法

设一个文法含有下列形式的产生式。

- 直接左递归

$$1) A \rightarrow A\beta \quad A \in V_N, \beta \in V_T$$

- 间接左递归

$$2) A \rightarrow B\beta$$

$$B \rightarrow A\alpha \quad A, B \in V_N, \alpha, \beta \in V_T$$

当文法是左递归时不能采用自顶向下分析法。

消除左递归

(1) 直接消除产生式中的左递归：假定关于非终结符P的规则为：

$$A \rightarrow A\alpha \mid \beta \quad \text{其中}\beta\text{不以}A\text{开头。}$$

可以把A的规则等价地改写为如下的非直接左递归形式：

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A' \mid \varepsilon$$

消除直接左递归

一般而言，假定 A 关于的全部产生式是

$$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \dots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$$

其中，每个 α 都不等于 ε ，每个 β 都不以 A 开头

那么，消除 A 的直接左递归性就是把这些规则改写成：

$$A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots \mid \beta_n A'$$

$$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \alpha_m A' \mid \varepsilon$$

消除直接左递归的示例

例：文法G(E):

$$E \rightarrow E + T \mid E - T \mid T$$

$$T \rightarrow T * F \mid T / F \mid F$$

$$F \rightarrow (E) \mid i$$

经消去左递归后变成:

$$E \rightarrow TE_1$$

$$E_1 \rightarrow +TE_1 \mid -TE_1 \mid \varepsilon$$

$$T \rightarrow FT_1$$

$$T_1 \rightarrow *FT_1 \mid /FT_1 \mid \varepsilon$$

$$F \rightarrow (E) \mid i$$

$$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \dots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$$

$$A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots \mid \beta_n A'$$

$$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \alpha_m A' \mid \varepsilon$$

消除间接左递归

① 非直接左递归

$$S \rightarrow Aa \mid b$$

$$A \rightarrow Sd \mid \varepsilon$$

② 先变换成直接左递归

$$S \rightarrow Aa \mid b$$

$$A \rightarrow Aad \mid bd \mid \varepsilon$$

③ 再消除左递归

$$S \rightarrow Aa \mid b$$

$$A \rightarrow bd A' \mid A'$$

$$A' \rightarrow adA' \mid \varepsilon$$

消除间接左递归的算法

- I. 把文法G的所有非终结符按任意顺序排列成 $A_1, A_2, \dots, A_n$;
- II. FOR $i:=1$ TO n {
 FOR $j:=1$ TO $i-1$ {
 把形如 $A_i \rightarrow A_j \gamma$ 的规则改写成
 $A_i \rightarrow \delta_1 \gamma | \delta_2 \gamma | \dots | \delta_k \gamma$; (其中 $A_j \rightarrow \delta_1 | \delta_2 | \dots | \delta_k$ 是关于 A_j 的所有规则)
 }
 消除关于 P_i 规则的直接左递归性
}
- III. 化简所得的文法。去除从开始符出发无法到达的非终结符的产生规则。

提取公共左因子

方法：假定关于A的规则是：

$$A \rightarrow \delta\beta_1 \mid \delta\beta_2 \mid \dots \mid \delta\beta_n \mid \gamma_1 \mid \gamma_2 \mid \dots \mid \gamma_m$$

(其中，每个 γ 不以 δ 开头)

那么，可以把这些规则改写成

$$A \rightarrow \delta A' \mid \gamma_1 \mid \gamma_2 \mid \dots \mid \gamma_m$$

$$A' \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$$

- 经过反复提取左因子，就能够把每个非终结符(包括新引进者)的所有候选首符集变成成为两两不相交。

提取公共左因子

■ 例：悬空 $else$ 的文法

$$\begin{aligned} stmt &\rightarrow \text{if } expr \text{ then } stmt \text{ else } stmt \\ &\quad | \text{if } expr \text{ then } stmt \\ &\quad | \text{other} \end{aligned}$$

提左因子

$$\begin{aligned} stmt &\rightarrow \text{if } expr \text{ then } stmt \text{ } otherpart \mid \text{other} \\ otherpart &\rightarrow \text{else } stmt \mid \varepsilon \end{aligned}$$

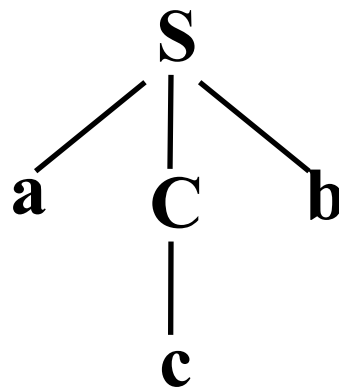
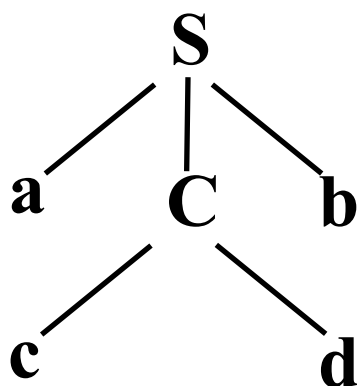
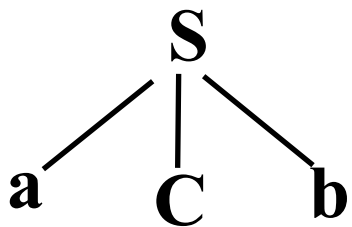
3、自上而下的语法分析

自上而下分析是一种语法分析方法，从文法的开始符号出发，尝试构建一个推导树，以验证输入符号串是否符合给定文法。自上而下分析在某些情况下是有效的。

例如：文法：

$$S \rightarrow aCb \quad C \rightarrow cd \mid c$$

为输入串 $w = acb$ 建立分析树



自上而下分析的局限性

- **左递归**：自上而下分析对于左递归文法处理起来比较困难。**左递归可能导致自上而下分析进入无限循环**，难以正确地完成分析，必须消除左递归。
- **左公因子的处理**：自上而下分析在处理文法中存在左公因子的情况时。**左公因子可能导致分析器无法确定选择哪个产生式进行推导**，影响分析的准确性。
- **可能存在回溯**：自上而下分析在**面对文法中存在多个可能的推导路径**时，可能会出现非确定性，**分析器可能需要尝试多个路径**，**在遇到错误时可能需要回溯到之前的状态重新尝试其他分析路径**，会增加分析的时间复杂度。

自上而下分析技术

- 自上而下的语法分析是一种从文法的开始符号出发，尝试构建一个推导过程，以匹配输入字符串的过程。在自上而下的语法分析中，通常会使用递归下降分析、LL分析、预测分析等方法。

递归下降分析

递归下降分析是一种常见的自顶向下的语法分析方法，用于判断一个输入字符串是否符合给定文法规则。在递归下降分析中，每个非终结符对应一个分析函数，通过递归调用这些函数来分析输入字符串。以下是递归下降分析方法的详细步骤：

- ① **准备工作**：首先需要准备好文法的产生式集合，改写文法消除左递归、提取公共表达等，每个非终结符对应编写一个分析函数。通常会有一个主函数作为入口，用于开始分析过程。
- ② **编写分析函数**：对于每个非终结符 A ，编写一个对应的分析函数 $A()$ 。在分析函数中，根据文法规则选择相应的产生式进行推导。
- ③ **递归调用**：在分析函数中，如果遇到一个非终结符 B ，则递归调用 $B()$ 函数进行分析；这样可以逐步展开并分析整个输入字符串。

递归下降分析（续）

- ④ **匹配终结符**：在分析函数中，如果遇到一个终结符（如标识符、关键字等），则需要**匹配输入字符串中的相应部分**，以确保输入符合语法规则。
- ⑤ **错误处理**：如果在分析过程中**发现输入字符串不符合语法规则**，需要进行错误处理，通常是报告语法错误并尝试恢复分析过程。
- ⑥ **递归下降分析**：通过递归调用各个分析函数，逐步分析整个输入字符串，**直到达到文法的终结符或无法继续推导**。

递归下降分析示例

例：文法G(E):

$$E \rightarrow E + T \mid E - T \mid T$$

$$T \rightarrow T * F \mid T / F \mid F$$

$$F \rightarrow (E) \mid i$$

经消去左递归后变成:

$$E \rightarrow TE_1$$

$$E_1 \rightarrow +TE_1 \mid -TE_1 \mid \varepsilon$$

$$T \rightarrow FT_1$$

$$T_1 \rightarrow *FT_1 \mid /FT_1 \mid \varepsilon$$

$$F \rightarrow (E) \mid i$$

递归下降分析示例（续）

$E \rightarrow TE_1$

$E_1 \rightarrow +TE_1 \mid -TE_1 \mid \varepsilon$

$T \rightarrow FT_1$

$T_1 \rightarrow *FT_1 \mid /FT_1 \mid \varepsilon$

$F \rightarrow (E) \mid i$

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <ctype.h>
4
5  char input[100];
6  int pos = 0;
7
8  void E();
9  void E1();
10 void T();
11 void T1();
12 void F();
13
14 void error();
15 void advance();
16 void match(char expected);
17
```

```
18 void E()
19 {
20     advance();
21     T();
22     E1();
23 }
24
25 void E1()
26 {
27     advance();
28     if (input[pos] == '+')
29     {
30         match('+');
31         T();
32         E1();
33     }
34     else if (input[pos] == '-')
35     {
36         match('-');
37         T();
38         E1();
39     }
40     else
41     {
42         // Epsilon production
43     }
44 }
```

这是算术表达式文法的简易语法分析程序，每个非终结符对应一个处理函数，`error()`是错误处理函数，`advance()`是跳过输入串的空格，`match()`函数处理匹配运算符。

示例 (续)

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid -TE' \mid \varepsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid /FT' \mid \varepsilon$

$F \rightarrow (E) \mid i$

```
46 void T()  
47 {  
48     advance();  
49     F();  
50     T1();  
51 }  
52  
53 void T1()  
54 {  
55     advance();  
56     if (input[pos] == '*')  
57     {  
58         match('*');  
59         F();  
60         T1();  
61     }  
62     else if (input[pos] == '/')  
63     {  
64         match('/');  
65         F();  
66         T1();  
67     }  
68     else  
69     {  
70         // Epsilon production  
71     }  
72 }
```

```
74 void F()  
75 {  
76     advance();  
77     if (isdigit(input[pos]))  
78     {  
79         printf("%c", input[pos]);  
80         pos++;  
81     }  
82     else if (input[pos] == '(')  
83     {  
84         match('(');  
85         E();  
86         match(')');  
87     }  
88     else  
89     {  
90         error();  
91     }  
92 }  
93  
94 int main()  
95 {  
96     printf("Enter an arithmetic expression: ");  
97     fgets(input, sizeof(input), stdin);  
98     E();  
99     if (input[pos] == '\0')  
100     |     printf("\nInput successfully parsed\n");  
101     else  
102     |     error();  
103     return 0;  
104 }
```

示例（续）

$E \rightarrow TE_1$

$E_1 \rightarrow +TE_1 \mid -TE_1 \mid \varepsilon$

$T \rightarrow FT_1$

$T_1 \rightarrow *FT_1 \mid /FT_1 \mid \varepsilon$

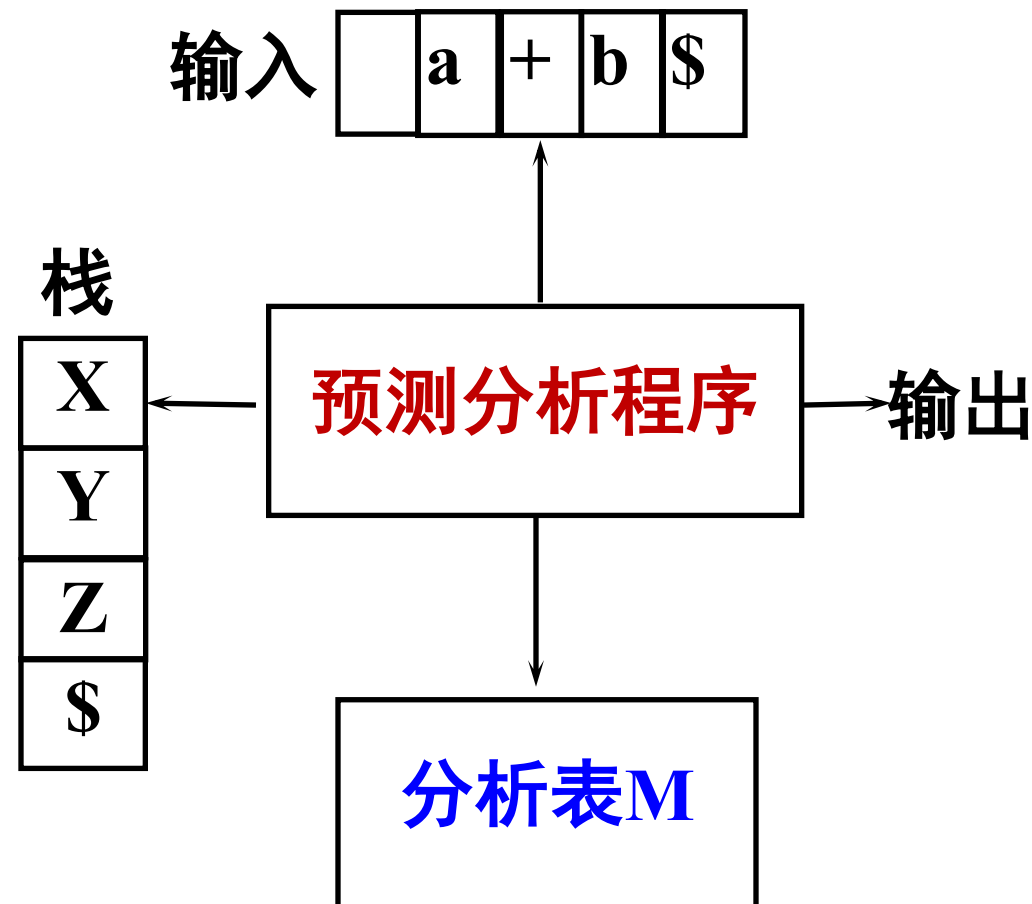
$F \rightarrow (E) \mid i$

```
C:\Windows\system32\cmd.e:  X  +  v
Enter an arithmetic expression: 3 *5 - (6 + 7 - 9 / 3)
3*5-(6+7-9/3)
Input successfully parsed
请按任意键继续 . . . |
```

```
106 void error()
107 {
108     printf("Error in parsing input\n");
109     exit(1);
110 }
111
112 void advance()
113 {
114     while (isspace(input[pos]))
115         pos++;
116 }
117
118 void match(char expected)
119 {
120     if (input[pos] == expected)
121     {
122         printf("%c", expected);
123         pos++;
124     }
125     else
126     {
127         error();
128     }
129 }
```

预测分析法

- 预测分析器（LL(1)分析器）的逻辑结构：
 - ① 通用的**控制程序**(表驱动预测分析程序)
 - ② 统一形式的LL(1)**预测分析表**M，不同文法使用内容不同但结构相同的分析表
 - ③ **分析栈**，存放文法符号，\$为栈底符号
 - ④ **输入缓冲区**存放输入串，\$为输入串结束符



预测分析表

- 预测分析表是一个二维表，行表示文法中的非终结符，列表示文法中的终结符（包括输入符号和结束符号\$）。
- 每个表项包含一个产生式或一个错误标记。产生式指示应该使用哪个产生式进行推导，错误标记表示分析过程中的错误。

非终结符	终结符							
	i	+	-	*	/	(	)	\$
E	$E \rightarrow TE_1$					$E \rightarrow TE_1$		
E ₁		$E_1 \rightarrow +TE_1$	$E_1 \rightarrow -TE_1$				$E_1 \rightarrow \epsilon$	$E_1 \rightarrow \epsilon$
T	$T \rightarrow FT_1$					$T \rightarrow FT_1$		
T ₁		$T_1 \rightarrow \epsilon$	$T_1 \rightarrow \epsilon$	$T_1 \rightarrow *FT_1$	$T_1 \rightarrow /FT_1$		$T_1 \rightarrow \epsilon$	$T_1 \rightarrow \epsilon$
F	$F \rightarrow i$					$F \rightarrow (E)$		

预测分析表的构建准备

预测分析表的构建需要进行文法的预处理,

- ① 计算First集
- ② 计算Follow集。

FIRST集的计算

对于文法G的所有非终结符的每个候选式,其首终结符集称为FIRST集:

$$\text{FIRST}(\alpha) = \{a \mid \alpha \Rightarrow^* a..., a \in V_T\}$$

若 $\alpha \Rightarrow^* \varepsilon$ 时, 规定 $\varepsilon \in \text{FIRST}(\alpha)$

FIRST(α): 从 α 可能推导出的所有首终结符号或 ε

求FIRST集的示例

【例】
 $S \rightarrow aAb$
 $A \rightarrow cd|c$



$\text{FIRST}(aAb) = \{a\}$
 $\text{FIRST}(cd) = \{c\}$
 $\text{FIRST}(c) = \{c\}$



$\text{FIRST}(S) = \{a\}$
 $\text{FIRST}(A) = \{c\}$

【例】
 $S \rightarrow Aa$
 $A \rightarrow a|\epsilon$



$\text{FIRST}(a) = \{a\}$
 $\text{FIRST}(\epsilon) = \{\epsilon\}$
 $\text{FIRST}(Aa) = \{a\}$



$\text{FIRST}(S) = \{a\}$
 $\text{FIRST}(A) = \{a, \epsilon\}$

构造FIRST集的算法

(1) 若 $\alpha = a\alpha'$, 且 $a \in V_T$, 则 $a \in \text{FIRST}(\alpha)$;

例: $\text{FIRST}(i) = \{i\}$

$\text{FIRST}(+TE_1) = \{+\}$

(2) 若 $\alpha = X\alpha'$, $X \in V_N$, 且有产生式 $X \rightarrow b\dots$, 则把 b 加入到 $\text{FIRST}(\alpha)$ 中;

例: $\text{FIRST}(FT_1) = \{ (, i \}$

$E \rightarrow TE_1$

$E_1 \rightarrow +TE_1 | -TE_1 | \varepsilon$

$T \rightarrow FT_1$

$T_1 \rightarrow *FT_1 | /FT_1 | \varepsilon$

$F \rightarrow (E) | i$

构造FIRST集的算法(续)

(3) 若 $\alpha = X_1 X_2 \dots X_n$, 其中 $X_i \in V_N$, $1 \leq i \leq n$;

- ① 将 $\text{FIRST}(X_1)$ 中的一切非 ε 的终结符加进 $\text{FIRST}(\alpha)$;
- ② 若 $\varepsilon \in \text{FIRST}(X_1)$, 则将 $\text{FIRST}(X_2)$ 中的一切非 ε 的终结符加进 $\text{FIRST}(\alpha)$;
- ③ 若 $\varepsilon \in \text{FIRST}(X_1)$ 且 $\varepsilon \in \text{FIRST}(X_2)$, 则将 $\text{FIRST}(X_3)$ 中的所有非 ε 的终结符加进 $\text{FIRST}(\alpha)$;
- ④ 依此类推, 若对于一切 $1 \leq i \leq n$, $\varepsilon \in \text{FIRST}(X_i)$, 则将 ε 加进 $\text{FIRST}(\alpha)$ 。

例: $\text{FIRST}(FT') = \text{FIRST}(F) - \{\varepsilon\} = \{ (, i \}$

$E \rightarrow TE_1$
 $E_1 \rightarrow +TE_1 \mid -TE_1 \mid \varepsilon$
 $T \rightarrow FT_1$
 $T_1 \rightarrow *FT_1 \mid /FT_1 \mid \varepsilon$
 $F \rightarrow (E) \mid i$

构造FIRST集的算法

$\text{FIRST}(\textcolor{green}{(E)}) = \{ (\}, \text{FIRST}(i) = \{ i \}$

$\text{FIRST}(F) = \{ (, i \}$

$\text{FIRST}(*FT_1) = \{ * \}, \text{FIRST}(/FT_1) = \{ / \}$

$\text{FIRST}(T_1) = \{ *, /, \epsilon \}$

$\text{FIRST}(FT_1) = \text{FIRST}(F) - \{ \epsilon \} = \{ (, i \}$

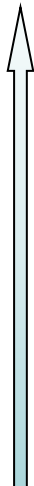
$\text{FIRST}(T) = \text{FIRST}(F) - \{ \epsilon \} = \{ (, i \}$

$\text{FIRST}(+TE_1) = \{ + \}, \text{FIRST}(-TE_1) = \{ - \}, \text{FIRST}(\epsilon) = \{ \epsilon \}$

$\text{FIRST}(E_1) = \{ +, -, \epsilon \}$

$\text{FIRST}(TE_1) = \text{FIRST}(T) - \{ \epsilon \} = \{ (, i \}$

$\text{FIRST}(E) = \text{FIRST}(T) - \{ \epsilon \} = \{ (, i \}$



$E \rightarrow TE_1$
 $E_1 \rightarrow +TE_1 | -TE_1 | \epsilon$
 $T \rightarrow FT_1$
 $T_1 \rightarrow *FT_1 | /FT_1 | \epsilon$
 $F \rightarrow (E) | i$

FOLLOW集的计算

对于文法G的非终结符的后继符号集称为FOLLOW集，定义如下：

$$\text{FOLLOW}(A) = \{a \mid S \Rightarrow^* \dots Aa\dots, a \in V_T\}$$

如果A是某个句型的最右符号,即: $S \Rightarrow^* \dots A$,

那么\$属于FOLLOW(A)

$E \Rightarrow^* T+TE_1$, 则 $+\in \text{FOLLOW}(T)$

$$E \rightarrow TE_1$$

$$E_1 \rightarrow +TE_1 \mid -TE_1 \mid \varepsilon$$

$$T \rightarrow FT_1$$

$$T_1 \rightarrow *FT_1 \mid /FT_1 \mid \varepsilon$$

$$F \rightarrow (E) \mid i$$

构造集合FOLLOW的算法

(1) 若A为开始符号, 则把 “\$”加入FOLLOW(A)中;

$$\$ \in \text{FOLLOW}(E)$$

(2) 若 $B \rightarrow \alpha A \beta$ ($\beta \neq \varepsilon$), 则把 $\text{FIRST}(\beta) - \{\varepsilon\}$ 加入FOLLOW(A)中;

$$\text{由 } F \rightarrow (E) \text{ 可知, }) \in \text{FOLLOW}(E)$$

(3) 若 $B \rightarrow \alpha A$ 或 $B \rightarrow \alpha A \beta$, 且 $\beta \Rightarrow^* \varepsilon$, 则把FOLLOW(B)加入FOLLOW(A) 中。

由 $E \rightarrow TE_1$, 应把FOLLOW(E)加入 $\in \text{FOLLOW}(E_1)$

由 $E_1 \rightarrow +TE_1$ 且 $E_1 \Rightarrow \varepsilon$, 应把FOLLOW(E_1)加入FOLLOW(T)

注: FOLLOW集合中不能有 ε

$$E \rightarrow TE_1$$

$$E_1 \rightarrow +TE_1 | -TE_1 | \varepsilon$$

$$T \rightarrow FT_1$$

$$T_1 \rightarrow *FT_1 | /FT_1 | \varepsilon$$

$$F \rightarrow (E) | i$$

求FOLLOW的示例

$E \rightarrow TE_1$

$E_1 \rightarrow +TE_1 | -TE_1 | \varepsilon$

$T \rightarrow FT_1$

$T_1 \rightarrow *FT_1 | /FT_1 | \varepsilon$

$F \rightarrow (E) | i$

$\text{FIRST}(F) = \{(, i\}$

$\text{FIRST}(T_1) = \{*, /, \varepsilon\}$

$\text{FIRST}(T) = \{(, i\}$

$\text{FIRST}(E_1) = \{+, -, \varepsilon\}$

$\text{FIRST}(E) = \{(, i\}$

$\because E$ 是开始符号 $\therefore \$ \in \text{FOLLOW}(E)$;

又 $F \rightarrow (E) \therefore) \in \text{FOLLOW}(E)$; $\text{FOLLOW}(E) = \{ \$,) \}$

$\because E \rightarrow TE_1 \therefore \text{FOLLOW}(E)$ 加入 $\text{FOLLOW}(E_1)$

$\text{FOLLOW}(E_1) = \{ \$,) \}$

$\because E_1 \rightarrow +TE_1 | -TE_1 \therefore \text{FIRST}(E_1) - \{\varepsilon\}$ 加入 $\text{FOLLOW}(T)$

又 $E_1 \Rightarrow \varepsilon, \therefore \text{FOLLOW}(E_1)$ 加入 $\text{FOLLOW}(T)$

$\text{FOLLOW}(T) = \{ +, -,), \$ \}$

$\because T \rightarrow FT_1 \therefore \text{FOLLOW}(T)$ 加入 $\text{FOLLOW}(T_1)$

$\text{FOLLOW}(T_1) = \text{FOLLOW}(T) = \{ +, -,), \$ \}$

$\because T \rightarrow FT_1 \therefore \text{FOLLOW}(F) = \text{FIRST}(T_1) - \{\varepsilon\}$

又 $T_1 \Rightarrow \varepsilon \therefore \text{FOLLOW}(T)$ 加入 $\text{FOLLOW}(F)$

$\text{FOLLOW}(F) = \{ *, +, -,), /, \$ \}$

预测分析表的构建方法

预测分析表的构建需要进行文法的预处理，包括计算First集和Follow集。

对于每个非终结符A和终结符a：

- ① 如果存在产生式 $A \rightarrow \alpha$ ，则对于 $\text{First}(\alpha)$ 中的每个终结符a，将产生式 $A \rightarrow \alpha$ 添加到表项 $M(A, a)$ 中。
- ② 如果 $\epsilon \in \text{First}(\alpha)$ ，则对于 $\text{Follow}(A)$ 中的每个终结符a，将产生式 $A \rightarrow \alpha$ 添加到表项 $M(A, a)$ 中。
- ③ 如果 $\epsilon \in \text{First}(\alpha)$ 且 $\$ \in \text{Follow}(A)$ ，将产生式 $A \rightarrow \alpha$ 添加到表项 $M(A, \$)$ 中。

表达式文法的预测分析表

$E \rightarrow TE_1$
 $E_1 \rightarrow +TE_1 | -TE_1 | \epsilon$
 $T \rightarrow FT_1$
 $T_1 \rightarrow *FT_1 | /FT_1 | \epsilon$
 $F \rightarrow (E) | i$

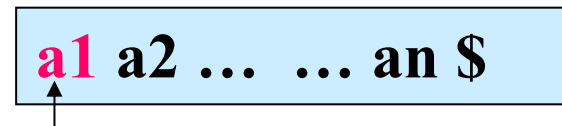
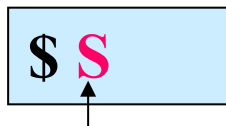
$FIRST(F) = \{ (, i \}$
 $FIRST(T_1) = \{ *, /, \epsilon \}$
 $FIRST(T) = \{ (, i \}$
 $FIRST(E_1) = \{ +, -, \epsilon \}$
 $FIRST(E) = \{ (, i \}$

$FOLLOW(E) = \{ \$,) \}$
 $FOLLOW(E_1) = \{ \$,) \}$
 $FOLLOW(T) = \{ +, -,), \$ \}$
 $FOLLOW(T_1) = \{ +, -,), \$ \}$
 $FOLLOW(F) = \{ *, +, -,), /, \$ \}$

非终结符	终结符							
	i	+	-	*	/	(	)	\$
E	$E \rightarrow TE_1$					$E \rightarrow TE_1$		
E ₁		$E_1 \rightarrow +TE_1$	$E_1 \rightarrow -TE_1$				$E_1 \rightarrow \epsilon$	$E_1 \rightarrow \epsilon$
T	$T \rightarrow FT_1$					$T \rightarrow FT_1$		
T ₁		$T_1 \rightarrow \epsilon$	$T_1 \rightarrow \epsilon$	$T_1 \rightarrow *FT_1$	$T_1 \rightarrow /FT_1$		$T_1 \rightarrow \epsilon$	$T_1 \rightarrow \epsilon$
F	$F \rightarrow i$					$F \rightarrow (E)$		

预测分析法的处理过程

- 在分析器启动时，**输入指针**指向输入串的第一个符号，**分析栈**中存放着栈底符号\$和文法的开始符号S,各指针置初值



- 分析的每一步都根据当前**栈顶符号X**和当前**输入符号a**查看**分析表项M[X,a]**，以决定相应的动作。
- 对任何(X, a)，执行**三种可能的动作**之一：
 - ① 若 $X=a='\$'$ ，则分析成功，**停止**分析。
 - ② 若 $X=a\neq '\$'$ ，即栈顶符号与当前扫描的输入符号a匹配，则将**X从栈顶逐出**，输入串指针指向**下一个输入符号**，继续分析下一个符号。

预测分析法的处理过程(续)

■ (3) 若X为非终结符号A，则查分析表项 $M[A,a]$ ：

- ① 若 $M[A,a]$ 中存放的是 $A \rightarrow y_1 y_2 \dots y_k$ ，则将A从栈顶逐出，让 $y_1, y_2, \dots, y_k$ 反序进栈。
- ② 若 $M[A,a]$ 中存放的是 $A \rightarrow \epsilon$ ，则将A逐出，不推什么进栈。
- ③ 若 $M[A,a]$ 中存放的是出错(空白)，则调用ERROR()进行错误处理,分析结束。

预测分析过程示例：输入串为：i+i*i\$

非终结符	终结符							
	i	+	-	*	/	(	)	\$
E	$E \rightarrow TE_1$					$E \rightarrow TE_1$		
E ₁		$E_1 \rightarrow +TE_1$	$E_1 \rightarrow -TE_1$				$E_1 \rightarrow \epsilon$	$E_1 \rightarrow \epsilon$
T	$T \rightarrow FT_1$					$T \rightarrow FT_1$		
T ₁		$T_1 \rightarrow \epsilon$	$T_1 \rightarrow \epsilon$	$T_1 \rightarrow *FT_1$	$T_1 \rightarrow /FT_1$		$T_1 \rightarrow \epsilon$	$T_1 \rightarrow \epsilon$
F	$F \rightarrow i$					$F \rightarrow (E)$		

步骤	符号栈	读入符号	剩余符号串	使用规则
1.	\$E	i	+i*i\$	
2.	\$E'T	i	+i*i\$	$E \rightarrow TE'$
3.	\$E'T'F	i	+i*i\$	$T \rightarrow FT'$
4.	\$E'T' i	i	+i*i\$	$F \rightarrow i$
5.	\$E'T'	+	i*i\$	(出栈，读下一个符号)
6.	\$E'	+	i*i\$	$T \rightarrow \epsilon$
7.	\$E'T+	+	i*i\$	$E \rightarrow +TE'$

继续分析：输入串为：i+i*i\$

非终结符	终结符							
	i	+	-	*	/	(	)	\$
E	$E \rightarrow TE_1$					$E \rightarrow TE_1$		
E ₁		$E_1 \rightarrow +TE_1$	$E_1 \rightarrow -TE_1$				$E_1 \rightarrow \epsilon$	$E_1 \rightarrow \epsilon$
T	$T \rightarrow FT_1$					$T \rightarrow FT_1$		
T ₁		$T_1 \rightarrow \epsilon$	$T_1 \rightarrow \epsilon$	$T_1 \rightarrow *FT_1$	$T_1 \rightarrow /FT_1$		$T_1 \rightarrow \epsilon$	$T_1 \rightarrow \epsilon$
F	$F \rightarrow i$					$F \rightarrow (E)$		

步骤	符号栈	读入符号	剩余符号串	使用规则
8·	\$E'T	i	*i\$	
9·	\$E'T'F	i	*i\$	$T \rightarrow FT'$
10·	\$E'T' i	i	*i\$	$F \rightarrow i$
11·	\$E'T'	*	i\$	
12·	\$E'T'F*	*	i\$	$T' \rightarrow *FT'$
13·	\$E'T'F	i	\$	

继续分析：输入串为：i+i*i\$

非终结符	终结符							
	i	+	-	*	/	(	)	\$
E	$E \rightarrow TE_1$					$E \rightarrow TE_1$		
E ₁		$E_1 \rightarrow +TE_1$	$E_1 \rightarrow -TE_1$				$E_1 \rightarrow \epsilon$	$E_1 \rightarrow \epsilon$
T	$T \rightarrow FT_1$					$T \rightarrow FT_1$		
T ₁		$T_1 \rightarrow \epsilon$	$T_1 \rightarrow \epsilon$	$T_1 \rightarrow *FT_1$	$T_1 \rightarrow /FT_1$		$T_1 \rightarrow \epsilon$	$T_1 \rightarrow \epsilon$
F	$F \rightarrow i$					$F \rightarrow (E)$		

步骤	符号栈	读入符号	剩余符号串	使用规则
14·	\$E'T' i	i	\$	$F \rightarrow i$
15·	\$E'T'	\$		
16·	\$E'	\$		$T' \rightarrow \epsilon$
17·	\$	\$		$E' \rightarrow \epsilon$

LL(1)文法

LL(1)文法是一种特殊的上下文无关文法，文法名的内在含义：

- ① **L**：从左到右扫描输入串；
- ② **L**：用左推导规则构建推导过程；
- ③ **1**：在每一步分析时，只需查看输入串中的一个符号和栈顶符号；如果在每一步分析时，需要查看输入串中的 k 个符号和栈顶的 k 个符号,则称为LL(k)文法。

LL(1)文法的特点

- ① LL(1)文法每个非终结符的每个产生式都有一个唯一的前缀；能够根据当前的非终结符和输入符号预测使用哪个产生式，避免回溯。
- ② LL(1)文法中的产生式的First集和Follow集之间两两不相交，或者如果有 ϵ 产生式，则这些产生式的First集和Follow集之间两两不相交。
- ③ LL(1)文法可以构建LL(1)预测分析表M，用于预测分析过程。

判断文法是LL(1)文法的条件

若一个文法满足以下条件，则称该文法G为LL(1)文法：

(1) 文法**不含左递归**；

(2) 对于每个非终结符A的各个候选式的**终结首符号集两两不相交**。即，如果 $A \rightarrow \alpha_1 | \alpha_2 | \dots | \alpha_n$ ，则 $\text{FIRST}(\alpha_i) \cap \text{FIRST}(\alpha_j) = \Phi$ ，其中 $1 \leq i, j \leq n$ ，且 $i \neq j$ 。

(3) 对于文法中每个非终结符A，**若它某个候选式的终结首符号集包含 ϵ** ，则 **$\text{FIRST}(A) \cap \text{FOLLOW}(A) = \Phi$**

LL(1)文法示例

■ 设文法G[S]: $S \rightarrow iA$

$A \rightarrow :i=e|e$

解：文法 不含左递归；没有 ε 产生式。

检查LL(1)文法的条件：

①· $FIRST(:i=e)=\{ : \}$, $FIRST(=e)=\{=\}$, 不交；

②· A的所有候选式的FIRST集都不含 ε 。

满足LL(1)文法的条件，**是LL(1)文法。**

非LL(1)文法的示例

- 设有文法G: $S \rightarrow LU$
 $L \rightarrow i:|\epsilon$
 $U \rightarrow i=e$

解：文法不含左递归；检查LL(1)文法条件：

- ①· L的各个候选式的FIRST集合互不交；
- ②· L有个候选式的FIRST集合含 ϵ ,

而 $FIRST(i:)=\{i\}$, $FOLLOW(L)=\{i\}$, 是相交的。

所以**G不是LL(1)文法**。

例：判断下面的文法是否为LL(1)文法？若不是，可否改写为LL(1)文法？

$$S \rightarrow (L) | a$$

$$L \rightarrow L, S | S$$

- 文法含有左递归，故不是LL(1)文法
- 改写文法：消除左递归

$$S \rightarrow (L) | a$$

$$L \rightarrow SL'$$

$$L' \rightarrow , SL' | \epsilon$$

	FIRST	FOLLOW
S	(a	\$,)
L	(a	)
L'	, ε	)

$$\text{FIRST}((L)) \cap \text{FIRST}(a) = \phi$$

$$\text{FIRST}(,SL') \cap \text{FOLLOW}(L') = \phi$$

	a	(	)	,	\$
S	$S \rightarrow a$	$S \rightarrow (L)$			
L	$L \rightarrow SL'$	$L \rightarrow SL'$			
L'			$L' \rightarrow \epsilon$	$L' \rightarrow ,SL'$	

二义性文法不是LL(1)文法

① if-else 文法G[S]: $S \rightarrow iESeS \mid iES \mid a$
 $E \rightarrow b$

② 提取左公因子改写文法，得到G'[S]:

$S \rightarrow iESS' \mid a$ $S' \rightarrow eS \mid \epsilon$ $E \rightarrow b$

④ 文法分析表

	i	e	a	b	\$
S	$S \rightarrow iESS'$		$S \rightarrow a$		
S'		$S' \rightarrow eS$ $S' \rightarrow \epsilon$			$S' \rightarrow \epsilon$
C				$E \rightarrow b$	

③ 计算: $FIRST(iESS') = \{i\}$
 $FIRST(a) = \{a\}$
 $FIRST(eS) = \{e\}$
 $FIRST(S') = \{e, \epsilon\}$
 $FOLLOW(S') = \{e, \$\}$
 $FIRST(b) = \{b\}$

- a) 如果存在产生式 $A \rightarrow \alpha$ ，则对于 $First(\alpha)$ 中的每个终结符 a ，将产生式 $A \rightarrow \alpha$ 添加到表项 $M(A, a)$ 中。
- b) 如果 $\epsilon \in First(\alpha)$ ，则对于 $Follow(A)$ 中的每个终结符 a ，将产生式 $A \rightarrow \alpha$ 添加到表项 $M(A, a)$ 中。

因此，该文法不是LL(1)文法

LL(1)文法的分析：采用预测分析法

1. 构建LL(1)的预测分析表，行表示非终结符，列表示终结符（包括非空输入符和结束符\$），每个表项包含一个产生式或错误标记。
2. 初始化分析栈，将结束符\$入栈后，然后文法开始符进栈；重复以下步骤直到分析完成：
 - ① 从栈顶取出一个符号X，如果X是终结符且与当前输入符号匹配，则弹出栈顶符号和输入符号，继续下一个符号。
 - ② 如果X是非终结符，查找LL(1)分析表的 $M(X, a)$ 的表项，a是当前输入符号。
 - ③ 如果表项中有产生式 $A \rightarrow \beta$ ，则将 β 逆序入栈。
 - ④ 如果表项中是错误标记，则报错并结束分析。

自下而上分析法

- ① 自下而上语法分析是一种基于推导的语法分析方法，它从输入字符串开始，逐步将输入字符串规约为起始符号，通过构建语法树或分析栈来实现。
- ② 通过重复执行移进和归约操作，分析栈逐步演变，直到最终只包含起始符号；如果在分析过程中，最终分析栈只包含起始符号，并且输入字符串已经被完全处理，则分析成功，输入字符串符合文法。
- ③ 自下而上的分析方法中，LR分析器是一种常用的分析器类型，包括LR(0)、SLR、LR(1)、LALR等；LR分析器通过构建LR自动机和分析表来实现自下而上的语法分析，具有较强的表达能力和效率。

移进-归约

- ① 自下而上语法分析从输入字符串的左端开始，逐步将输入符号移入分析栈。
移进操作将输入符号移入分析栈，并继续处理下一个输入符号。
- ② 当分析栈顶部的符号序列匹配一个文法的右侧产生式时，可以执行归约操作；归约操作将栈顶的一些符号替换为一个非终结符，这个非终结符是产生式的左侧符号。

移进-归约的示例

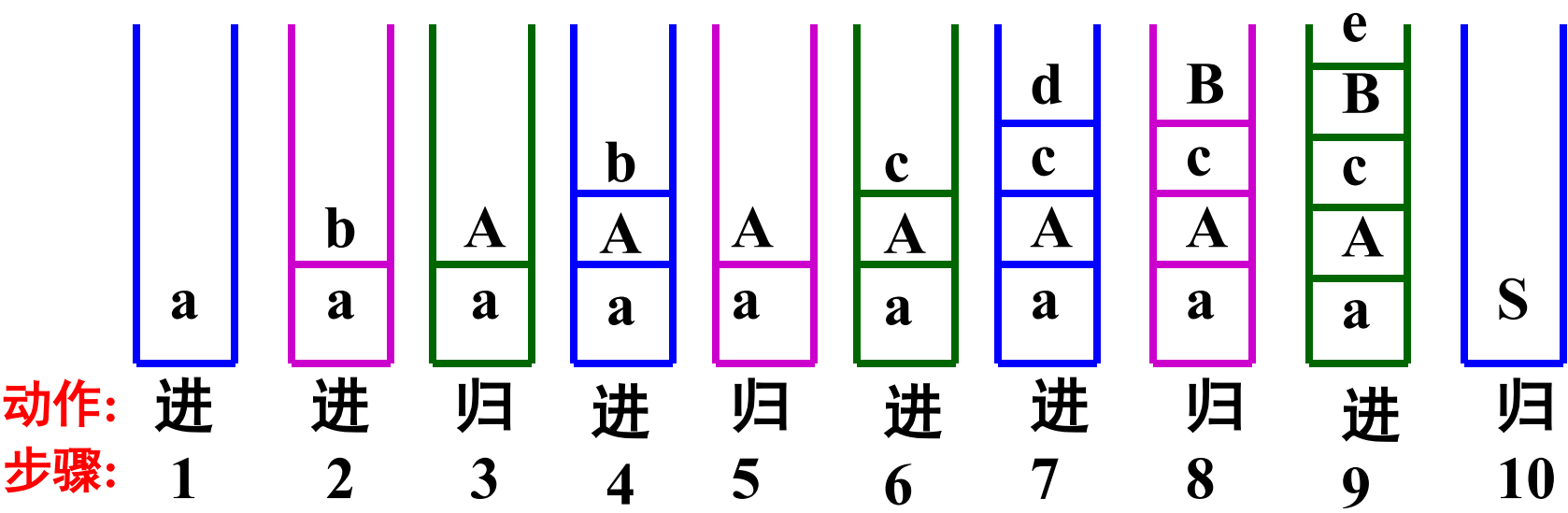
举例：已知文法G为(1) $S \rightarrow aAcBe$

(2) $A \rightarrow b$

(3) $A \rightarrow Ab$

(4) $B \rightarrow d$ 及输入串**abbcd**e

归约过程如下：



LR语法分析技术

① LR(k)语法分析：是一种有效的自下而上的语法分析方法，

L——从左向右扫描输入串； R——反向构造最右推导序列

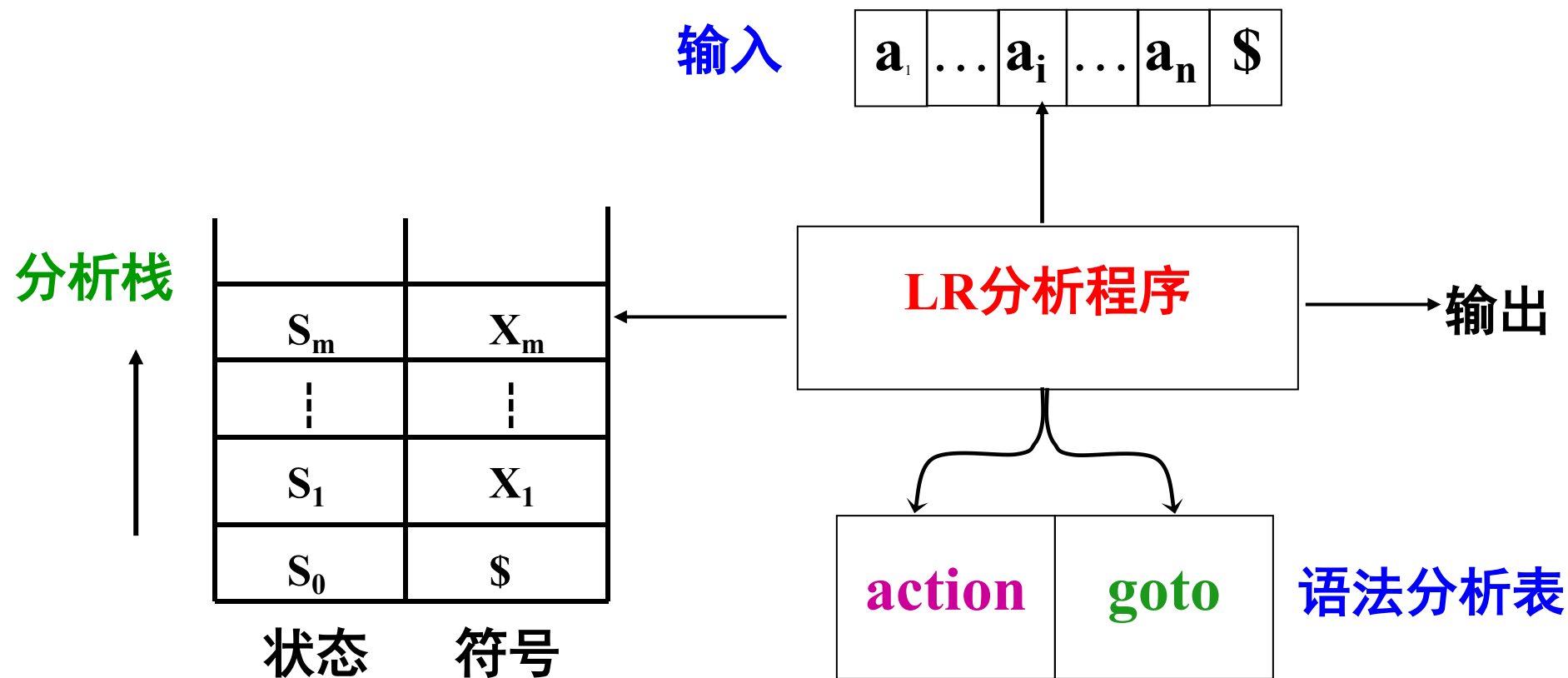
② K表示在做出语法分析决定移进或规约时，需要向前看 $K(K \geq 0)$ 个符号。

③ 向前看0个符号，就是LR(0)分析法；向前看1个符号，就是LR(1)分析法。

④ 大多数用上下文无关文法描述的语法结构，都可用LR分析器来识别。

⑤ LR分析器的分析过程可用LR自动机来描述。

LR语法分析器模型



LR分析器的模型

LR分析表的结构

- ACTION表**: $ACTION[k,a]$: 当栈顶状态为 k 且输入终结符为 a 时应采取的动作:
 - 移进**(s_n): 将输入符号 a 移入分析栈, 并转移到下一个状态 n 。
 - 归约**(r_k): 使用某个文法产生式 k 进行规约, 即把栈顶与第 k 个产生式右部匹配的符号串出栈, 再将产生式左部非终结符进栈。
 - 接受**(accept): 成功完成语法分析, 输入符合文法规则。
 - 报错**(Error): 分析表空白项, 表示分析器遇到了无法处理的错误。
- GOTO表**: 每个单元格包含在规约时确定规约后应该转移到的状态。

状态	ACTION (动作)			GOTO (转换)
	a	b	\$	A
0	S2			1
1			accept	
2	S2	S5		3
3		S4		
4	r1	r1	r1	
5	r2	r2	r2	

LR(0)文法

- 在LR(0)文法中，每个项目只包含文法产生式的左部和右部，只考虑每个项目的当前状态，不包含向前看符号。LR(0)项的形式为 $[A \rightarrow \alpha \bullet \beta]$ ，表示已分析到产生式 $A \rightarrow \alpha \beta$ 的某个位置。

项与LR(0)自动机

- 在文法的每一个产生式的右部添加一个圆点指示识别的位置，圆点右移一位代表LR(0)的一个项目（一个产生式项的个数为右部符号个数+1）。
- 例：产生式 $A \rightarrow XYZ$ 有四个项目：
 - $A \rightarrow \cdot XYZ$ 表明希望从输入看见由XYZ推出的串
 - $A \rightarrow X \cdot YZ$ 表明在输入中看到了X推出的串，并希望看到由YZ推导出的串
 - $A \rightarrow XY \cdot Z$ 表明在输入中看到了XY推出的串，并希望看到由Z推导出的串
 - $A \rightarrow XYZ \cdot$ 表明在输入中看到了XYZ推出的串，希望规约为A；称为规约项目
- 直观上说，项指明了在分析过程的某个时刻能看到产生式的多大部分。
- 由多个项构成的集合，称为项集；每个项集对应LR(0)自动机的状态节点。

项目的分类

- ① 归约项目: $A \rightarrow \alpha \bullet$
- ② 移进项目: $A \rightarrow \alpha \bullet a \beta, a \in V_T$
- ③ 待约项目: $A \rightarrow \alpha \bullet B \beta, B \in V_N$
- ④ 接受项目: $S' \rightarrow \alpha \bullet, S'$ 为文法开始符号。

构造LR(0)文法项集的方法

1. 新添产生式： $S' \rightarrow S$ ， S' 是增广文法的起始符号， S 是文法的起始符号；
2. 初始项集是从文法的开始符号构造的初始状态集合，包含文法的起始产生式的项目，即将点号放在产生式右部的开头位置。
3. 计算项集 I 的闭包 $CLOSURE(I)$:
 - ① I 的任何项都属于 $CLOSURE(I)$ ；
 - ② 若 $A \rightarrow \alpha \cdot X \beta$ 属于 $CLOSURE(I)$ ，那么，对任何 X 的产生式 $X \rightarrow \gamma$ ，项目 $X \rightarrow \cdot \gamma$ 也属于 $CLOSURE(I)$ ；
 - ③ 重复执行上述两步骤直至 $CLOSURE(I)$ 不再增大为止。

构造文法项集的示例

有拓展文法：

$S' \rightarrow E$

$E \rightarrow E + T$

$E \rightarrow T$

$T \rightarrow T * F$

$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow id$

初始项集 $I_0 = \{ S' \rightarrow \cdot E \}$

计算项集 I_0 的闭包 $CLOSURE(I_0)$ 得：

$I_0: S' \rightarrow \cdot E$

$E \rightarrow \cdot E + T$

$E \rightarrow \cdot T$

$T \rightarrow \cdot T * F$

$T \rightarrow \cdot F$

$F \rightarrow \cdot (E)$

$F \rightarrow \cdot id$

该项集 I_0 对应 LR(0) 自动机的初始状态节点！

GOTO函数及LR(0)自动机

- 给定LR(0)自动机中的状态 I 和输入符号 X ，GOTO(I, X) 函数表示状态 I 经过输入符号 X 后的转移状态。
- 对于状态 I 中的每个项目 $[A \rightarrow \alpha \cdot X \beta]$ ，其中 X 是一个文法符号，将点号向后移动一个位置得到新项目 $[A \rightarrow \alpha X \cdot \beta]$ ，这个项目将会出现在新状态 J 中，然后计算 J 的闭包，即 $\text{GOTO}(I, X) = \text{CLOSURE}(J)$ 。
- 新状态是GOTO(I, X) 的计算结果，即状态 I 经过输入符号 X 后的转移状态。

构造项集规范族

(1) 令LR(0)自动机的初态为 I_0 ,求其CLOSURE(I_0),得到初态项目集。即:

求CLOSURE($\{S' \rightarrow \cdot S\}$)

(2) 对所得项目集I和文法G的每个文法符号X(包括 V_T 和 V_N)

计算 $GO(I, X) = CLOSURE(J)$, 得到新的项目集。

其中 $J = \{\text{任何形如 } A \rightarrow \alpha X \cdot \beta \text{ 的项目} \mid A \rightarrow \alpha \cdot X \beta \text{ 属于 } I\}$

(3) 重复(1), 直至没有新的项集出现。

经过以上步骤构造出的项集的全体即为LR(0)项集规范族。

GOTO函数计算项集簇示例

I₀: $S' \rightarrow \cdot E$
 $E \rightarrow \cdot E + T$
 $E \rightarrow \cdot T$
 $T \rightarrow \cdot T * F$
 $T \rightarrow \cdot F$
 $F \rightarrow \cdot (E)$
 $F \rightarrow \cdot id$

计算GOTO(I₀,E)得I₁:

$S' \rightarrow E \cdot$
 $E \rightarrow E \cdot + T$

计算GOTO(I₀,T)得I₂:

$E \rightarrow T \cdot$
 $T \rightarrow T \cdot * F$

计算GOTO(I₀,F)得I₃:

$T \rightarrow F \cdot$

计算GOTO(I₀,())得I₄:

$F \rightarrow (\cdot E)$
 $E \rightarrow \cdot E + T$
 $E \rightarrow \cdot T$
 $T \rightarrow \cdot T * F$
 $T \rightarrow \cdot F$
 $F \rightarrow \cdot (E)$
 $F \rightarrow \cdot id$

计算GOTO(I₀,id)得I₅:

$F \rightarrow id \cdot$

计算GOTO(I₁,+)得I₆:

$E \rightarrow E + \cdot T$
 $T \rightarrow \cdot T * F$
 $T \rightarrow \cdot F$
 $F \rightarrow \cdot (E)$
 $F \rightarrow \cdot id$

计算GOTO(I₂,*)得I₇:

$T \rightarrow T * \cdot F$
 $F \rightarrow \cdot (E)$
 $F \rightarrow \cdot id$

计算GOTO(I₄,E)得I₈:

$F \rightarrow (E \cdot)$
 $E \rightarrow E \cdot + T$

计算GOTO(I₄,T)得I₂:

$E \rightarrow T \cdot$
 $T \rightarrow T \cdot * F$

计算GOTO(I₄,F)得I₃:

$T \rightarrow F \cdot$

计算GOTO(I₄,())得I₄:

计算GOTO(I₄,id)得I₅:

GOTO函数计算示例(续)

I_6

$E \rightarrow E + \cdot T$
 $T \rightarrow \cdot T * F$
 $T \rightarrow \cdot F$
 $F \rightarrow \cdot (E)$
 $F \rightarrow \cdot id$

计算GOTO(I_6, T)得 I_9 :

计算GOTO(I_6, F)得 I_3 :

计算GOTO($I_6, ($)得 I_4 :

计算GOTO(I_6, id)得 I_5 :

$E \rightarrow E + T \cdot$
 $T \rightarrow T \cdot * F$

I_7

$T \rightarrow T * \cdot F$
 $F \rightarrow \cdot (E)$
 $F \rightarrow \cdot id$

计算GOTO(I_7, F)得 I_{10} :

计算GOTO($I_7, ($)得 I_4 :

计算GOTO(I_7, id)得 I_5 :

$T \rightarrow T * F \cdot$

I_8

$F \rightarrow (E \cdot)$
 $E \rightarrow E \cdot + T$

计算GOTO($I_8,)$)得 I_{11} :

计算GOTO($I_8, +$)得 I_6 :

$F \rightarrow (E) \cdot$

内核项: 是指项目中的“.”符号位于产生式右部的非最左侧位置, 表示在推导过程中某些符号已经被处理过。

但是, 初始产生式 $S' \rightarrow \cdot E$ 是内核项。

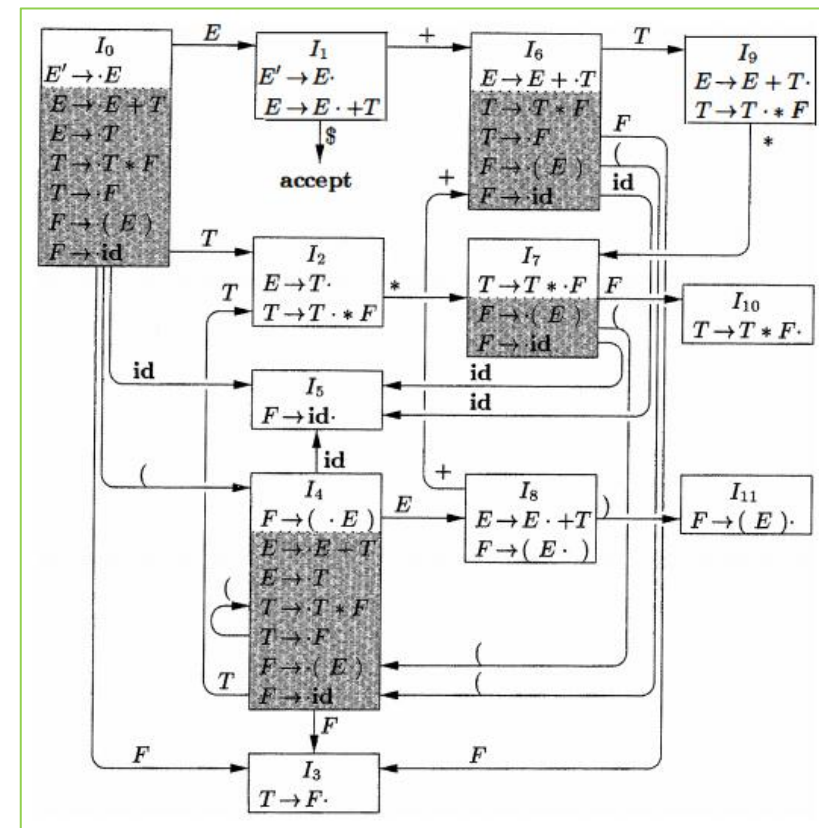
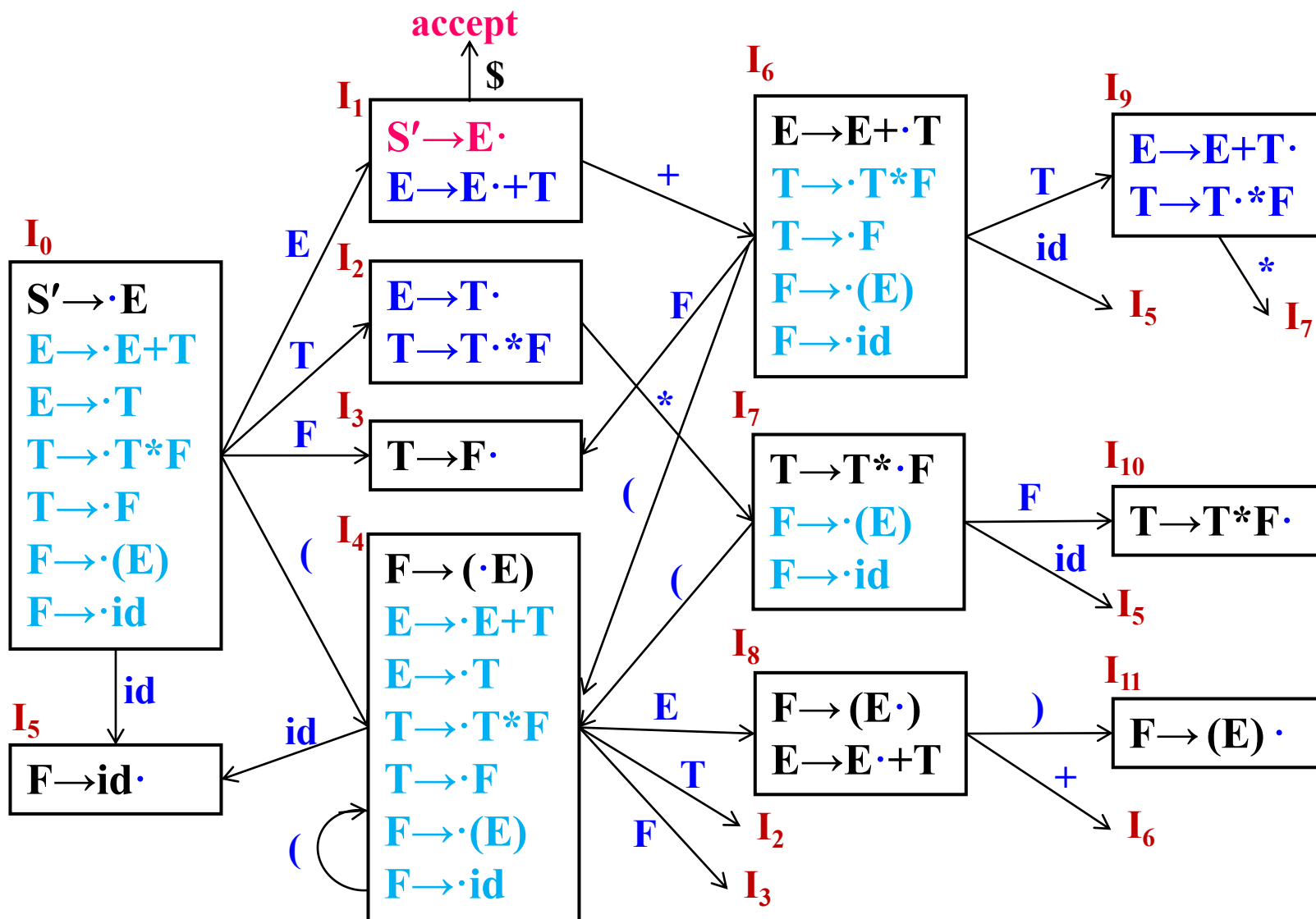
内核项是LR(0)自动机中状态的核心部分, 用于确定状态的转移和规约操作。

例如, 产生式 $E \rightarrow E + \cdot T$ 是内核项。

非内核项: 是指项目中的“.”符号位于产生式右部的最左侧, LR(0)语法分析中并不直接影响状态的转移和规约操作, 可由CLOSURE(I)计算得到。

例如, 产生式 $T \rightarrow \cdot T * F$ 是非内核项。

在前面的基础上构造LR(0)自动机

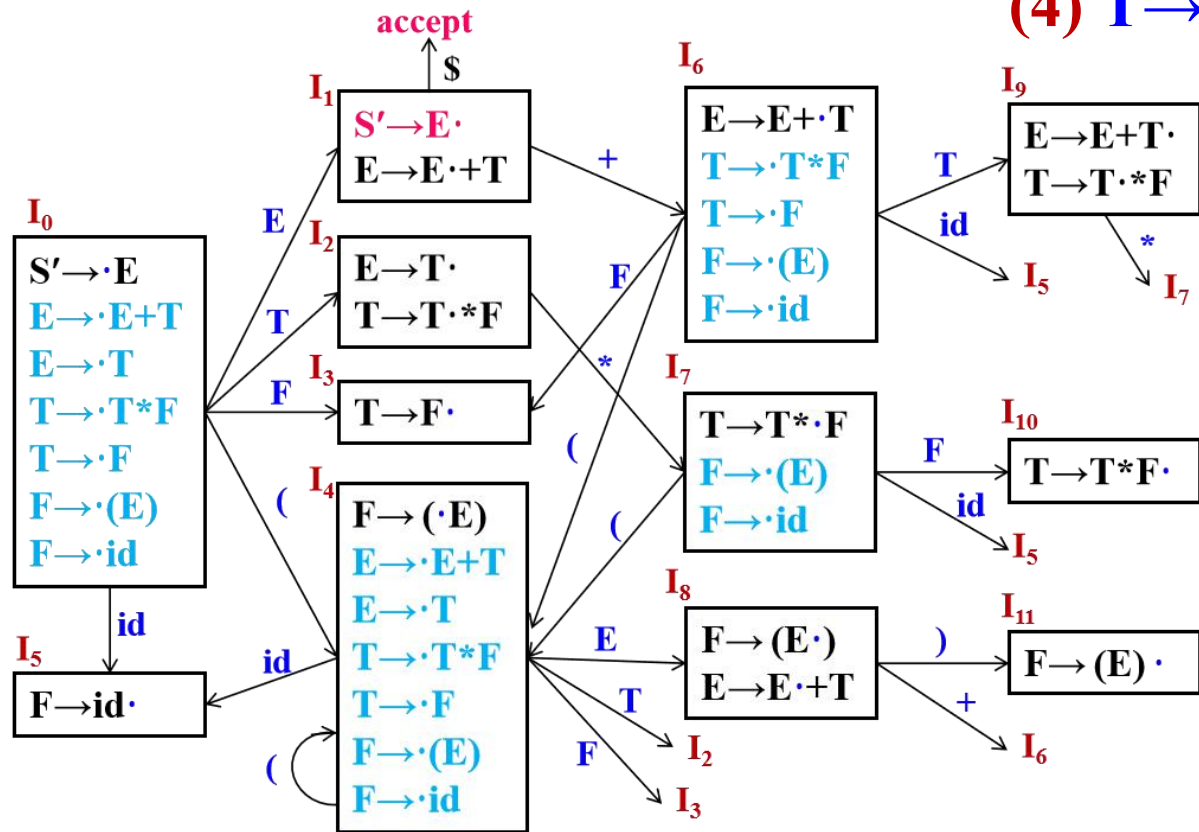


构造LR(0)分析表的算法

1. 若项目 $A \rightarrow \alpha \cdot X \beta$ 属于 I_k , 且 $GOTO(I_k, X) = I_j$:
 - ① 若 $X \in V_T$, 则置 $ACTION[k, X]$ 为“把 (j, X) 移进栈”, 简记为 S_j ;
 - ② 若 $X \in V_N$, 则置 $GOTO[k, X] = j$, j 为状态号;
2. 若项目 $A \rightarrow \alpha \cdot$ 属于 I_k , 则任何终结符 a 或结束符 $\$$, 置 $ACTION[k, a]$ 为“用产生式 $A \rightarrow \alpha$ 进行归约”, 简记为 r_i , i 为产生式序号;
3. 若项目 $S' \rightarrow S \cdot$ 属于 I_k , 则置 $ACTION[k, \$] = \text{accept}$ 为接受;
4. 对分析表中的空白处, 均置为“error”。

构造LR(0)分析表的示例

有产生式: (0) $S' \rightarrow E$ (1) $E \rightarrow E+T$ (2) $E \rightarrow T$ (3) $T \rightarrow T * F$
 (4) $T \rightarrow F$ (5) $F \rightarrow (E)$ (6) $F \rightarrow id$



状态	ACTION						GOTO		
	id	+	*	(	)	\$	E	T	F
0	s ₅			s ₄			1	2	3
1	r ₀	s ₆ /r ₀	r ₀	r ₀	r ₀	accept			
2	r ₂	r ₂	s ₇ /r ₂	r ₂	r ₂	r ₂			
3	r ₄	r ₄	r ₄	r ₄	r ₄	r ₄			
4	s ₅			s ₄			8	2	3
5	r ₆	r ₆	r ₆	r ₆	r ₆	r ₆			
6	s ₅			s ₄				9	3
7	s ₅			s ₄					10
8		s ₆			s ₁₁				
9	r ₁	r ₁	s ₇ /r ₁	r ₁	r ₁	r ₁			
10	r ₃	r ₃	r ₃	r ₃	r ₃	r ₃			
11	r ₅	r ₅	r ₅	r ₅	r ₅	r ₅			

注意: 该LR(0)分析表有移进-规约冲突。

因此, 该文法不属于LR(0)文法

LR(0)语法分析过程的一般步骤：

1. **构建LR(0)自动机：**根据文法构建LR(0)自动机，包括初始状态、状态扩展、状态转移等。
2. **初始化：**将初始状态压入分析栈中，并读入输入符号串的第一个符号。
3. **状态转移和规约，**重复以下步骤直到分析完成：
从分析栈顶取出当前状态，并根据当前输入符号a和自动机转移边。
 - ① 如果当前状态有转出边a，则将输入符号a移入符号栈，并将转向的状态压入状态栈。
 - ② 如果是规约操作，则根据规约表中的产生式进行规约操作，将产生式右部符号弹出，将产生式左部符号压入分析栈，并根据新的状态和产生式左部符号在LR(0)分析表中查找对应的动作。
 - ③ 如果是接受操作，则分析成功，输入符号串符合文法。
4. **错误处理：**如果在分析过程中遇到无法识别的输入符号或无法进行移入或规约操作的情况，可能会发生错误。此时需要进行错误处理，通常是报告语法错误或尝试恢复分析过程。

基于LR(0)分析表的语法分析器工作示例

假设输入串id*id

状态	ACTION						GOTO		
	id	+	*	(	)	\$	E	T	F
0	s ₅			s ₄			1	2	3
1	r ₀	s ₆ /r ₀	r ₀	r ₀	r ₀	accept			
2	r ₂	r ₂	s ₇ /r ₂	r ₂	r ₂	r ₂			
3	r ₄	r ₄	r ₄	r ₄	r ₄	r ₄			
4	s ₅			s ₄			8	2	3
5	r ₆	r ₆	r ₆	r ₆	r ₆	r ₆			
6	s ₅			s ₄				9	3
7	s ₅			s ₄					10
8		s ₆			s ₁₁				
9	r ₁	r ₁	s ₇ /r ₁	r ₁	r ₁	r ₁			
10	r ₃	r ₃	r ₃	r ₃	r ₃	r ₃			
11	r ₅	r ₅	r ₅	r ₅	r ₅	r ₅			

步骤	状态栈	符号栈	输入串	动作
1	0	\$	id*id\$	移入，转移到5
2	0 5	\$id	*id\$	按照F→id规约
3	0 3	F	*id\$	按照T→F规约
4	0 2	T	*id\$	移入，转移到7
5	0 2 7	T*	id\$	移入，转移到5
6	0 2 7 5	T*id	\$	根据F→id规约
7	0 2 7 10	T*F	\$	根据T→T*F规约
8	0 2	T	\$	根据E→T规约
9	0 1	E	\$	接受

注意：这里采用移进优先规约的操作策略。

LR(0)中的项目冲突

在LR(0)项集族中，

- ① 假如有形如 $I=\{X\rightarrow\alpha\cdot b\beta, A\rightarrow\gamma\cdot\}$ 的项集时，则在该项集中含有移进-归约冲突；
- ② 假如有形如 $I=\{A\rightarrow\gamma\cdot, B\rightarrow\delta\cdot\}$ 的项集时，则在该项集中含有归约-归约冲突。
- ③ 假如有形如 $I=\{X\rightarrow\alpha\cdot b\beta, A\rightarrow\gamma\cdot, B\rightarrow\delta\cdot\}$ 的项集时，则在该项集中同时含有移进-归约冲突和归约-归约冲突。

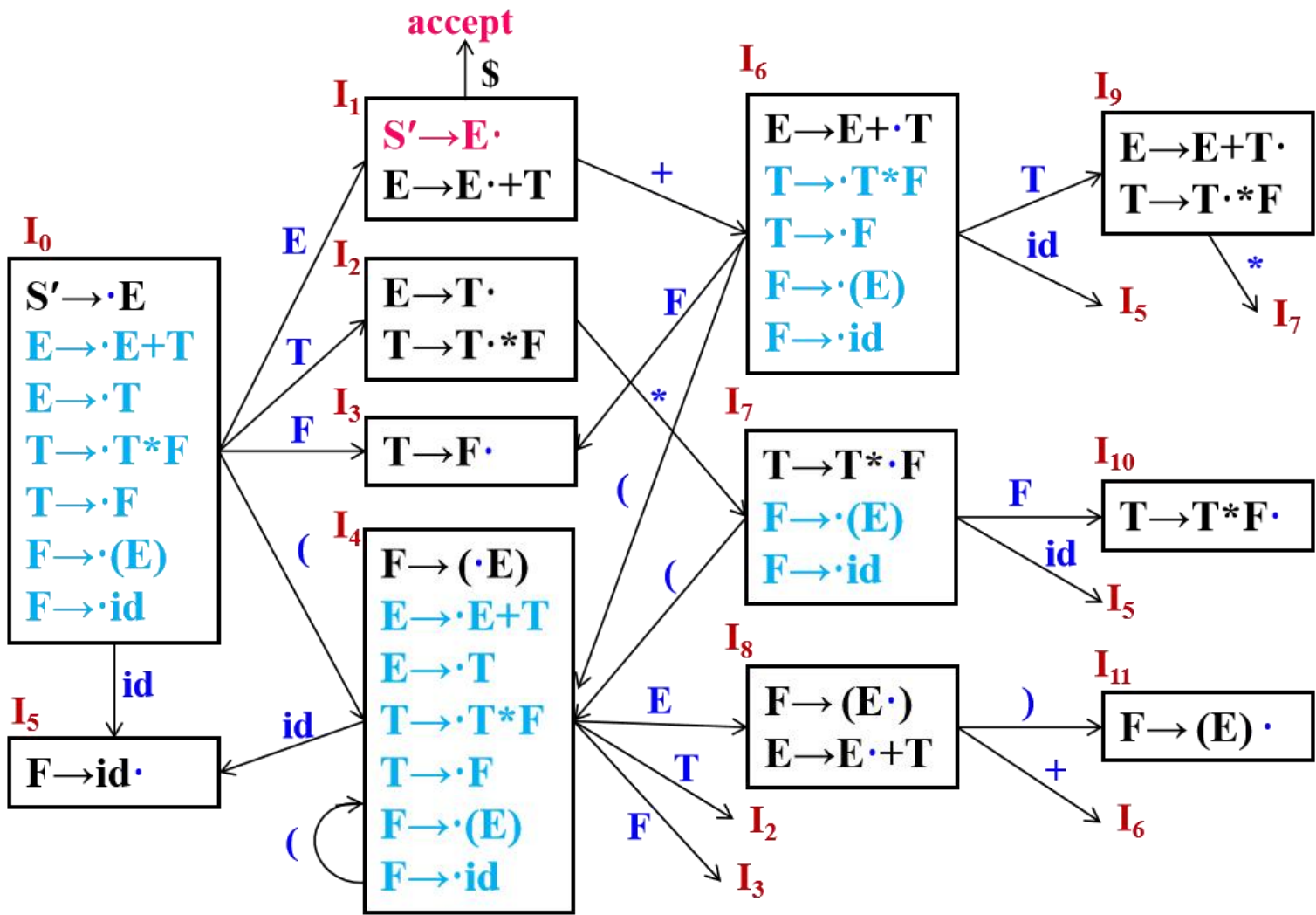
特别注意：如果LR(0)项集簇中的项集，存在移进-规约冲突或者存在规约-规约冲突，则该文法一定不是LR(0)文法。

LR(0)文法存在冲突的示例

有拓展产生式

$S' \rightarrow E$
 $E \rightarrow E + T$
 $E \rightarrow T$
 $T \rightarrow T * F$
 $T \rightarrow F$
 $F \rightarrow (E)$
 $F \rightarrow i$

文法的LR(0)自动机



I_1, I_2, I_9 中存在移进-归约冲突。

冲突的解决方法

假定LR(0)规范族中含有项集 $I=\{X\rightarrow\alpha\cdot b\beta, A\rightarrow\gamma\cdot, B\rightarrow\delta\cdot\}$

在该项集中含有移进-归约冲突和归约-归约冲突。

其中 $\alpha, \beta, \gamma, \delta$ 为文法符号串， b 为终结符。只要在所有含有 A 或 B 的句型中满足

① $\text{FOLLOW}(A) \cap \text{FOLLOW}(B) = \emptyset,$

② $\text{FOLLOW}(A) \cap \{b\} = \emptyset$

③ $\text{FOLLOW}(B) \cap \{b\} = \emptyset$

若在状态 I 时面临的输入符号为 a ，则动作可由下规定决策：

① 若 $a=b$ ，则移进。

② 若 $a \in \text{FOLLOW}(A)$ ，则用产生式 $A \rightarrow \gamma$ 进行归约；

③ 若 $a \in \text{FOLLOW}(B)$ ，则用产生式 $B \rightarrow \delta$ 进行归约；

④ 此外，报错。

冲突的解决方法

若LR(0)规范族含有m个移进项目

$$A_1 \rightarrow \alpha_1 \cdot a_1 \beta_1, A_1 \rightarrow \alpha_2 \cdot a_2 \beta_2, \dots, A_m \rightarrow \alpha_m \cdot a_m \beta_m$$

同时含有n个归约项目

$$B_1 \rightarrow \gamma_1 \cdot, B_2 \rightarrow \gamma_2 \cdot, \dots, B_n \rightarrow \gamma_n \cdot$$

则如果集合 $\{a_1, a_2, \dots, a_m\}$ 和 $\text{FOLLOW}(B_1), \text{FOLLOW}(B_2), \dots, \text{FOLLOW}(B_n)$ 两两交集都为空，那么上述规则仍可用于解决LR(0)规范族中的冲突。

- 1) 若 $a \in \{a_1, a_2, \dots, a_m\}$ ，则移进；
- 2) 若 $a \in \text{FOLLOW}(B_i)$ ， $i=1, 2, \dots, n$ ，则用 $B_i \rightarrow \gamma_i$ 进行归约；
- 3) 此外，报错。

SLR(1) 语法分析方法

- ① SLR(1)表示“Simple LR parser for a single lookahead token”，意为“简单的左到右解析”，是自底向上的语法分析方法；
- ② 由于SLR(1)文法考虑了Follow集合，因此可以在规约时更准确地确定使用哪个产生式进行规约，减少规约冲突的可能性。相比之下，而LR(0)文法则不考虑Follow集合，只根据项目的状态转移来构建分析表，LR(0)文法可能会存在更多的规约冲突。
- ③ SLR(1)文法通常比LR(0)文法更强大，可以覆盖更多类型的文法；LR(0)文法更容易构建，但是在某些文法可能无法准确地进行语法分析。

SLR(1) 分析表构造算法

算法：构造SLR分析表

输入：一个拓广文法 G'

输出： G' 的SLR分析表的函数 $action$ 和 $goto$

1· 构造 G' 的LR(0)项集族 $C = \{I_0, I_2, \dots, I_n\}$ 。

2· 对于状态 I_i 的分析动作如下：

(a) 若 $A \rightarrow \alpha \cdot aB \in I_i$ 且 $goto(I_i, a) = I_j$ 则 $action[i, a] = "s_j"$ ，表示移进状态j；

(b) 若 $A \rightarrow \alpha \cdot \in I_i$ ，对于所有 $a \in FOLLOW(A)$ 则 $action[i, a] = "r_k"$ ，表示按照产生式k规约，k为 $A \rightarrow \alpha$ 的编号， $A \neq S'$ ；

(c) 若 $S' \rightarrow S \cdot \in I_i$ ， $action[i, \$] = "accept"$

3· 若 $goto(I_i, A) = I_j$ ， $A \in V_N$ ，则 $goto[i, A] = j$

4· 分析表其余位置为 $error$

SLR(1)分析表的构造

有产生式: (0) $S' \rightarrow E$ (1) $E \rightarrow E+T$ (2) $E \rightarrow T$ (3) $T \rightarrow T * F$ (4) $T \rightarrow F$ (5) $F \rightarrow (E)$ (6) $F \rightarrow id$

在此复习FOLLOW集的计算方法

- (1) 若A为开始符号, 则把“\$”加入FOLLOW(A)中;
- (2) 若 $B \rightarrow \alpha A \beta$ ($\beta \neq \epsilon$), 则把 $FIRST(\beta) - \{\epsilon\}$ 加入FOLLOW(A)中;
- (3) 若 $B \rightarrow \alpha A$ 或 $B \rightarrow \alpha A \beta$, 且 $\beta \Rightarrow^* \epsilon$, 则把FOLLOW(B)加入FOLLOW(A)中。

$$FOLLOW(S') = \{ \$ \}$$

$$FOLLOW(E) = \{ +,), \$ \}$$

$$FOLLOW(T) = \{ *, +,), \$ \}$$

$$FOLLOW(F) = FOLLOW(T) = \{ *, +,), \$ \}$$

SLR(1)分析表的构造

$FOLLOW(S') = \{ \$ \}$ $FOLLOW(E) = \{ +,), \$ \}$

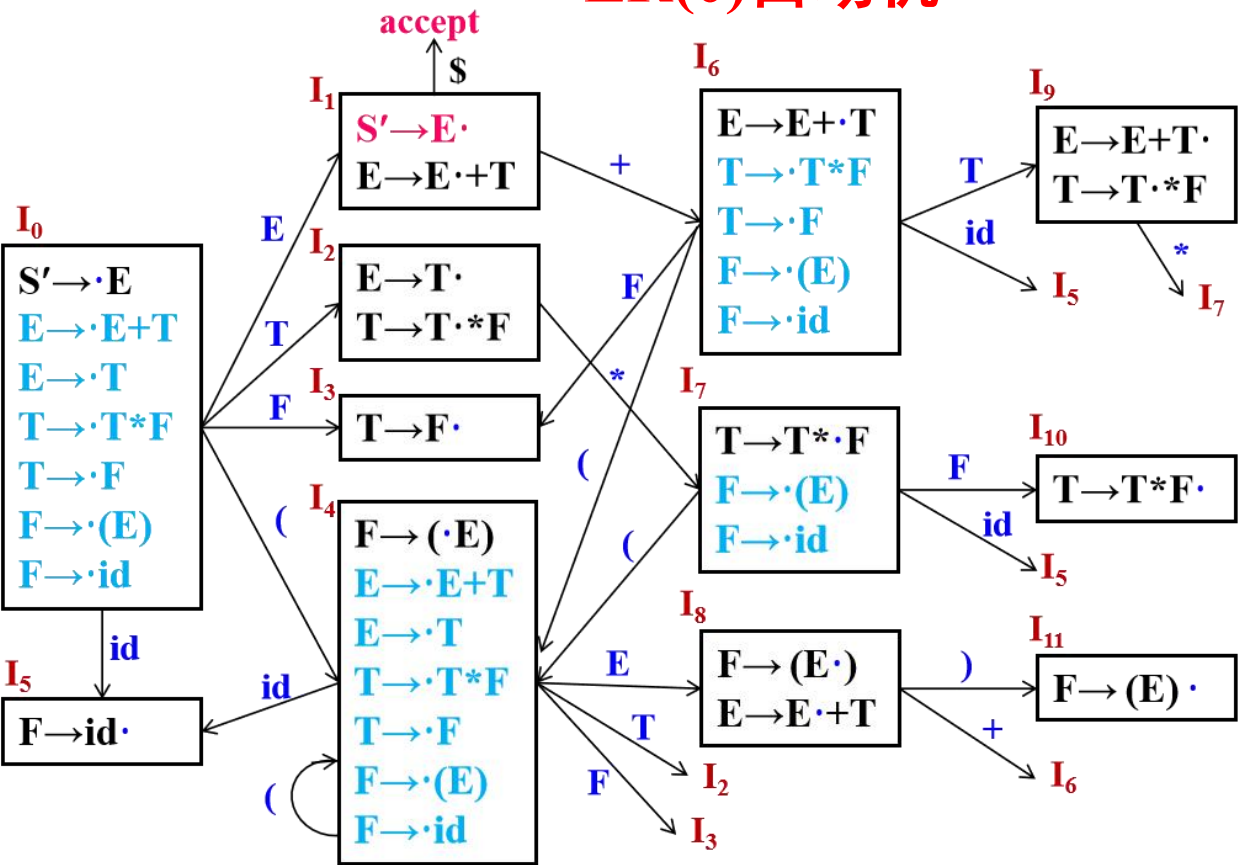
$FOLLOW(F) = FOLLOW(T) = \{ *, +,), \$ \}$

有产生式: (0) $S' \rightarrow E$

(1) $E \rightarrow E + T$ (2) $E \rightarrow T$ (3) $T \rightarrow T * F$

(4) $T \rightarrow F$ (5) $F \rightarrow (E)$ (6) $F \rightarrow id$

LR(0)自动机

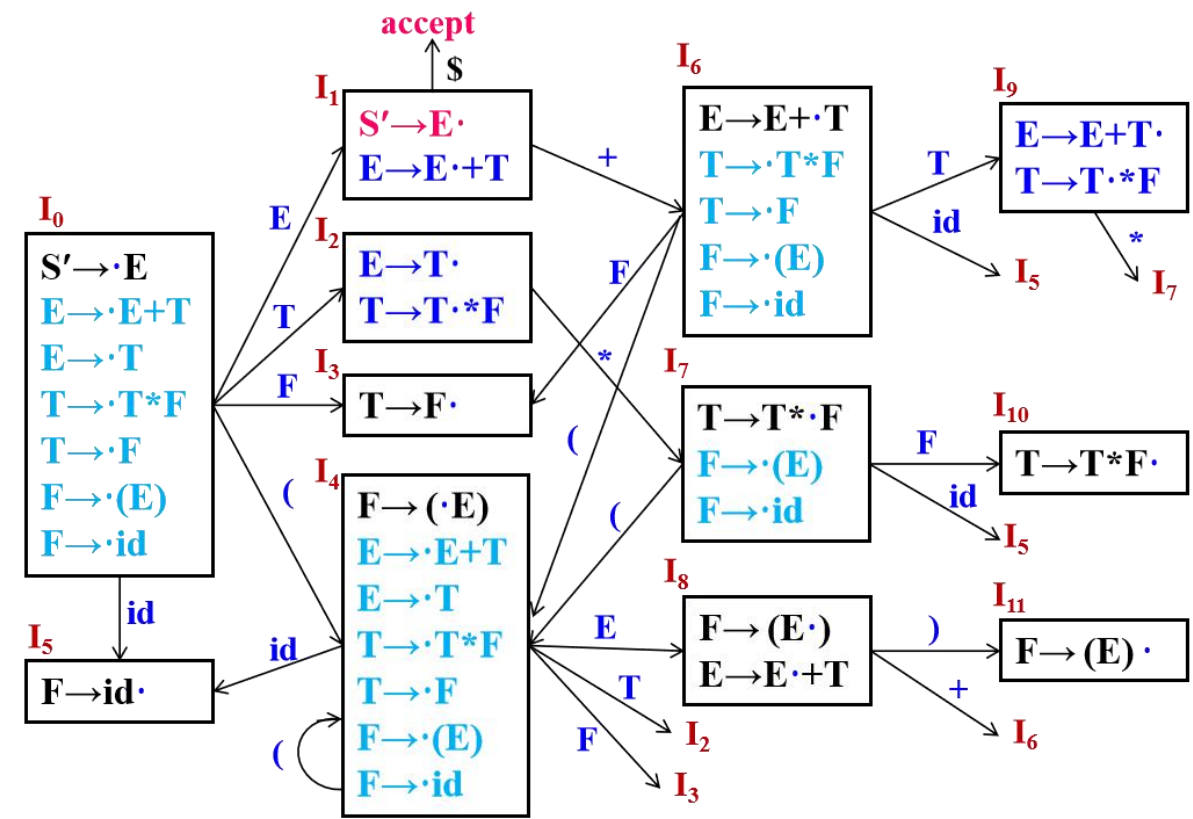


SLR(1)分析表

状态	ACTION						GOTO		
	id	+	*	(	)	\$	E	T	F
0	s ₅			s ₄			1	2	3
1		s ₆				accept			
2		r ₂	s ₇		r ₂	r ₂			
3		r ₄	r ₄		r ₄	r ₄			
4	s ₅			s ₄			8	2	3
5		r ₆	r ₆		r ₆	r ₆			
6	s ₅			s ₄				9	3
7	s ₅			s ₄					10
8		s ₆			s ₁₁				
9		r ₁	s ₇		r ₁	r ₁			
10		r ₃	r ₃		r ₃	r ₃			
11		r ₅	r ₅		r ₅	r ₅			

判断文法是否是SLR(1)文法的示例

表达式拓广文法：(0) $S' \rightarrow E$ (1) $E \rightarrow E+T$ (2) $E \rightarrow T$ (3) $T \rightarrow T * F$ (4) $T \rightarrow F$ (5) $F \rightarrow (E)$ (6) $F \rightarrow i$



在 I_1 中: $S' \rightarrow E \cdot$ $E \rightarrow E \cdot +T$
故 $FOLLOW(S') \cap \{+\} = \{\$ \} \cap \{+\} = \emptyset$

在 I_2 中: $E \rightarrow T \cdot$ $T \rightarrow T \cdot * F$
 $FOLLOW(E) \cap \{*\} = \{+,), \$ \} \cap \{*\} = \emptyset$

在 I_9 中: $E \rightarrow E+T \cdot$ $T \rightarrow T \cdot * F$
 $FOLLOW(E) \cap \{*\} = \{+,), \$ \} \cap \{*\} = \emptyset$

由前面分析可知LR(0)文法在 I_1 , I_2 , I_9 三个项目集中存在的移进-归约冲突,可以用SLR(1)方法解决, 因此该文法是SLR(1)文法。

SLR(1)分析器的工作过程

1. **初始化**：将**初始状态**（通常是包含文法的起始符号的项目）**压入分析栈**中，并**读入输入符号串的第一个符号**。
2. **状态转移和规约**：重复以下步骤直到分析完成：
从分析栈顶**取出当前状态**，并根据**当前输入符号和当前状态**在SLR(1)分析表中查找对应的动作。
 - ① 如果是**移入操作**，则将**当前输入符号移入分析栈**，并将**新的状态压入分析栈**。
 - ② 如果是**规约操作**，则根据规约表中的产生式进行规约操作，**将产生式右部符号弹出**，**将产生式左部符号压入分析栈**，并根据**新的状态和产生式左部符号**在SLR(1)分析表中**查找对应的动作**。
 - ③ 如果是**接受操作**，则**分析成功**，输入符号串符合文法。

SLR(1)分析器的工作过程

3. **错误处理**：如果在分析过程中遇到无法识别的输入符号或无法进行移入或规约操作的情况，可能会发生错误。此时需要进行错误处理，通常是报告语法错误或尝试恢复分析过程。
4. **分析结果**：如果分析成功并且输入符号串符合文法，则分析过程结束，可以得到语法分析树或其他分析结果。如果分析失败或遇到错误，则需要进行相应的错误处理。

示例：分析id*id+id\$

有产生式： (0) $S' \rightarrow E$ (1) $E \rightarrow E+T$ (2) $E \rightarrow T$ (3) $T \rightarrow T * F$ (4) $T \rightarrow F$ (5) $F \rightarrow (E)$ (6) $F \rightarrow id$

状态	ACTION						GOTO		
	id	+	*	(	)	\$	E	T	F
0	s ₅			s ₄			1	2	3
1		s ₆				accept			
2		r ₂	s ₇		r ₂	r ₂			
3		r ₄	r ₄		r ₄	r ₄			
4	s ₅			s ₄			8	2	3
5		r ₆	r ₆		r ₆	r ₆			
6	s ₅			s ₄				9	3
7	s ₅			s ₄					10
8		s ₆			s ₁₁				
9		r ₁	s ₇		r ₁	r ₁			
10		r ₃	r ₃		r ₃	r ₃			
11		r ₅	r ₅		r ₅	r ₅			

步骤	栈	符号	输入串	动作
1	0		id*id+id\$	移入
2	0 5	id	*id+id\$	根据 $F \rightarrow id$ 规约
3	0 3	F	*id+id\$	根据 $T \rightarrow F$ 规约
4	0 2	T	*id+id\$	移入
5	0 2 7	T*	id+id\$	移入
6	0 2 7 5	T*id	+id\$	根据 $F \rightarrow id$ 规约
7	0 2 7 10	T*F	+id\$	根据 $T \rightarrow T * F$ 规约
8	0 2	T	+id\$	根据 $E \rightarrow T$ 规约
9	0 1	E	+id\$	移入
10	0 1 6	E+	id\$	移入
11	0 1 6 5	E+id	\$	根据 $F \rightarrow id$ 规约
12	0 1 6 3	E+F	\$	根据 $T \rightarrow F$ 规约
13	0 1 6 9	E+T	\$	根据 $F \rightarrow E+T$ 规约
14	0 1	E	\$	接受

SLR(1)文法是无二义性的，但无二义性文法可能不是SLR(1)文法的例子

文法产生式： 拓广文法 G' 的LR(0)项集族为：

$S \rightarrow L=R$

$S \rightarrow R$

$L \rightarrow *R$

$L \rightarrow \text{id}$

$R \rightarrow L$

$I_0:$ $S' \rightarrow \cdot S$
 $S \rightarrow \cdot L=R$
 $S \rightarrow \cdot R$
 $L \rightarrow \cdot *R$
 $L \rightarrow \cdot \text{id}$
 $R \rightarrow \cdot L$

$I_1:$ $S' \rightarrow S \cdot$

$I_2:$ $S \rightarrow L \cdot =R$
 $R \rightarrow L \cdot$

$I_3:$ $S \rightarrow R \cdot$

$I_4:$ $L \rightarrow * \cdot R$
 $R \rightarrow \cdot L$
 $L \rightarrow \cdot *R$
 $L \rightarrow \cdot \text{id}$

$I_5:$ $L \rightarrow \text{id} \cdot$

$I_6:$ $S \rightarrow L = \cdot R$
 $R \rightarrow \cdot L$
 $L \rightarrow \cdot *R$
 $L \rightarrow \cdot \text{id}$

$I_7:$ $L \rightarrow *R \cdot$

$I_8:$ $R \rightarrow L \cdot$

$I_9:$ $S \rightarrow L = R \cdot$

构造该文法的有限自动机

文法产生式:

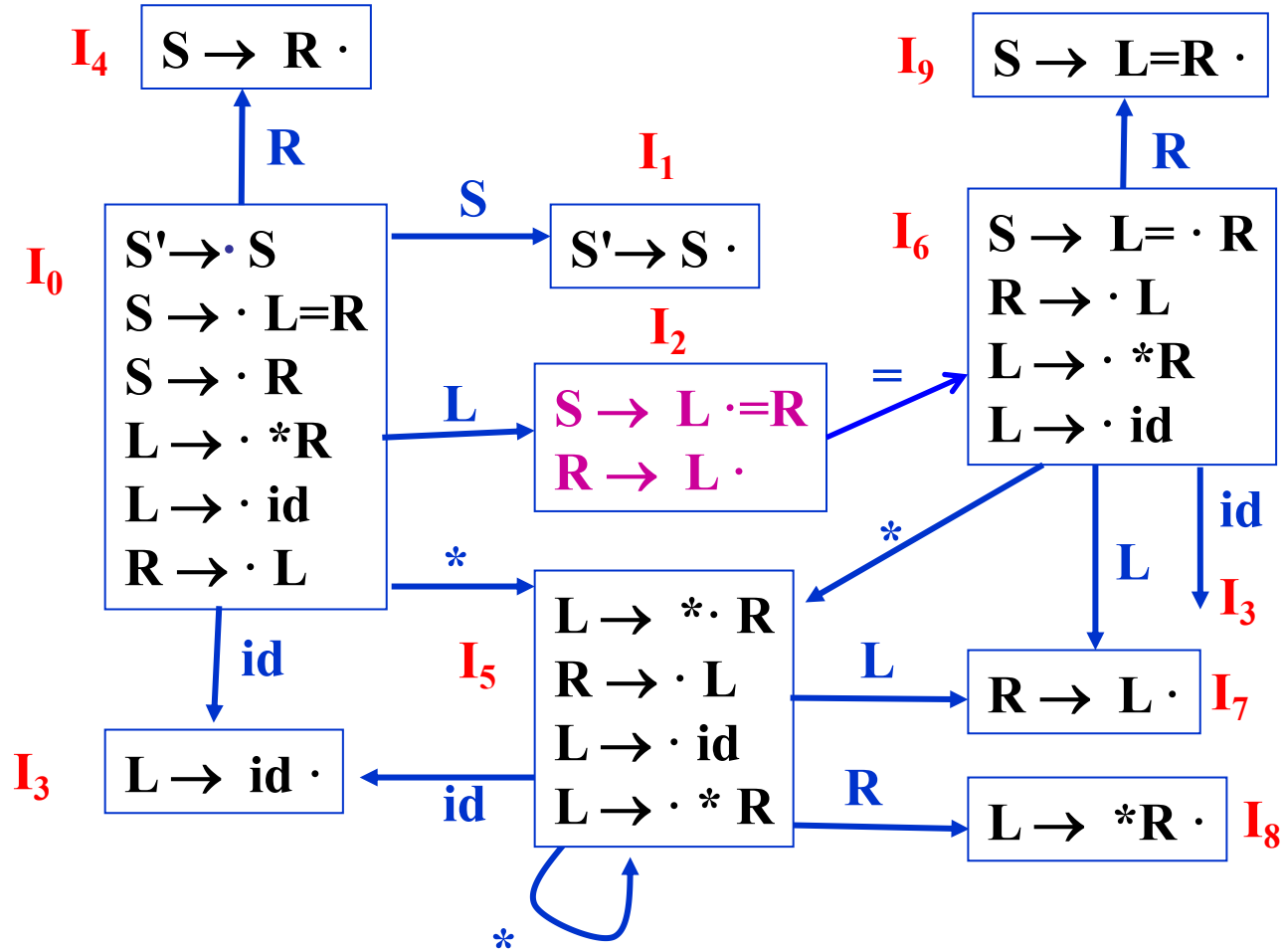
$$S \rightarrow L=R$$

$$S \rightarrow R$$

$$L \rightarrow *R$$

$$L \rightarrow \text{id}$$

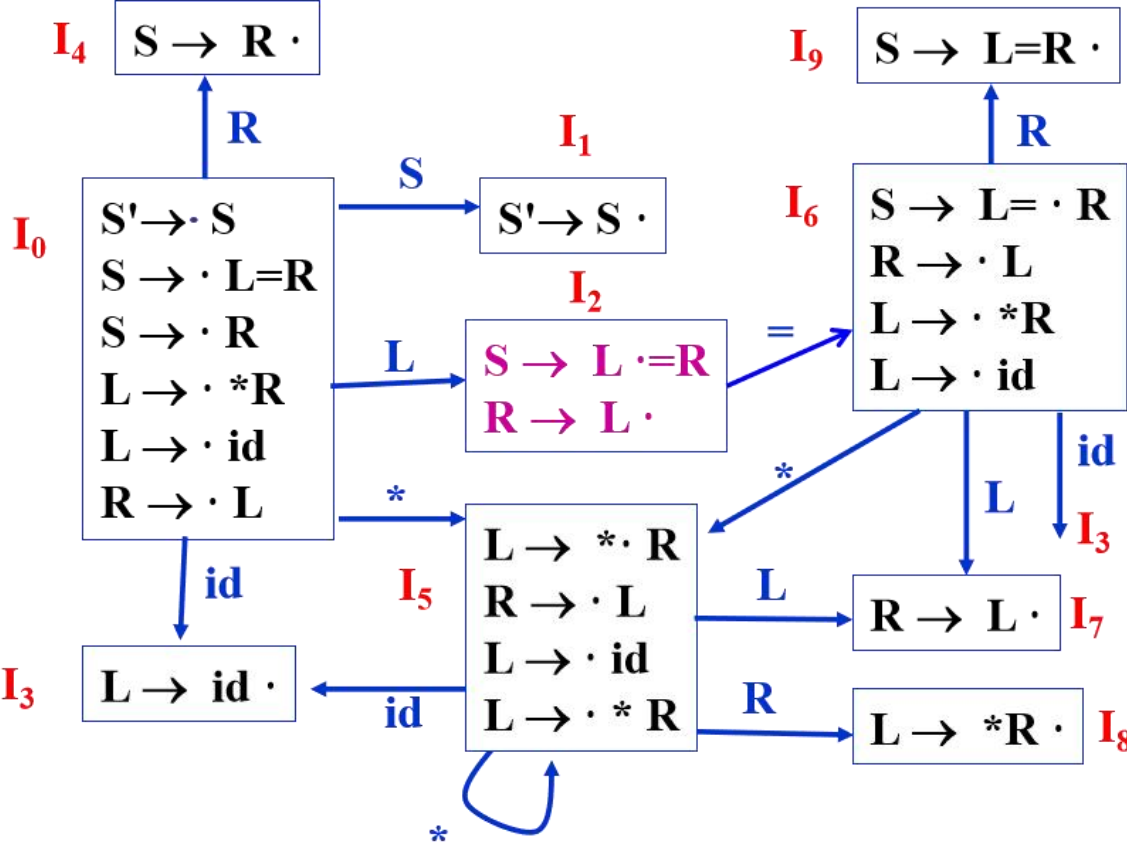
$$R \rightarrow L$$



构造SLR(1)语法分析表

文法产生式:

$S' \rightarrow S \quad S \rightarrow L=R \quad S \rightarrow R$
 $L \rightarrow *R \quad L \rightarrow id \quad R \rightarrow L$



$\text{follow}(S') = \text{follow}(S) = \{\$, \$\}$

$\text{follow}(L) = \text{follow}(R) = \{=, \$\}$

状态	ACTION				GOTO		
	id	=	*	\$	S	L	R
0	s ₃		s ₅		1	2	4
1				accept			
2		s ₆ /r ₅		r ₅			
3		r ₄		r ₄			
4		r ₂		r ₂			
5	s ₃		s ₅			7	8
6	s ₃		s ₅			7	9
7		r ₅		r ₅			
8		r ₃		r ₃			
9		r ₁		r ₁			

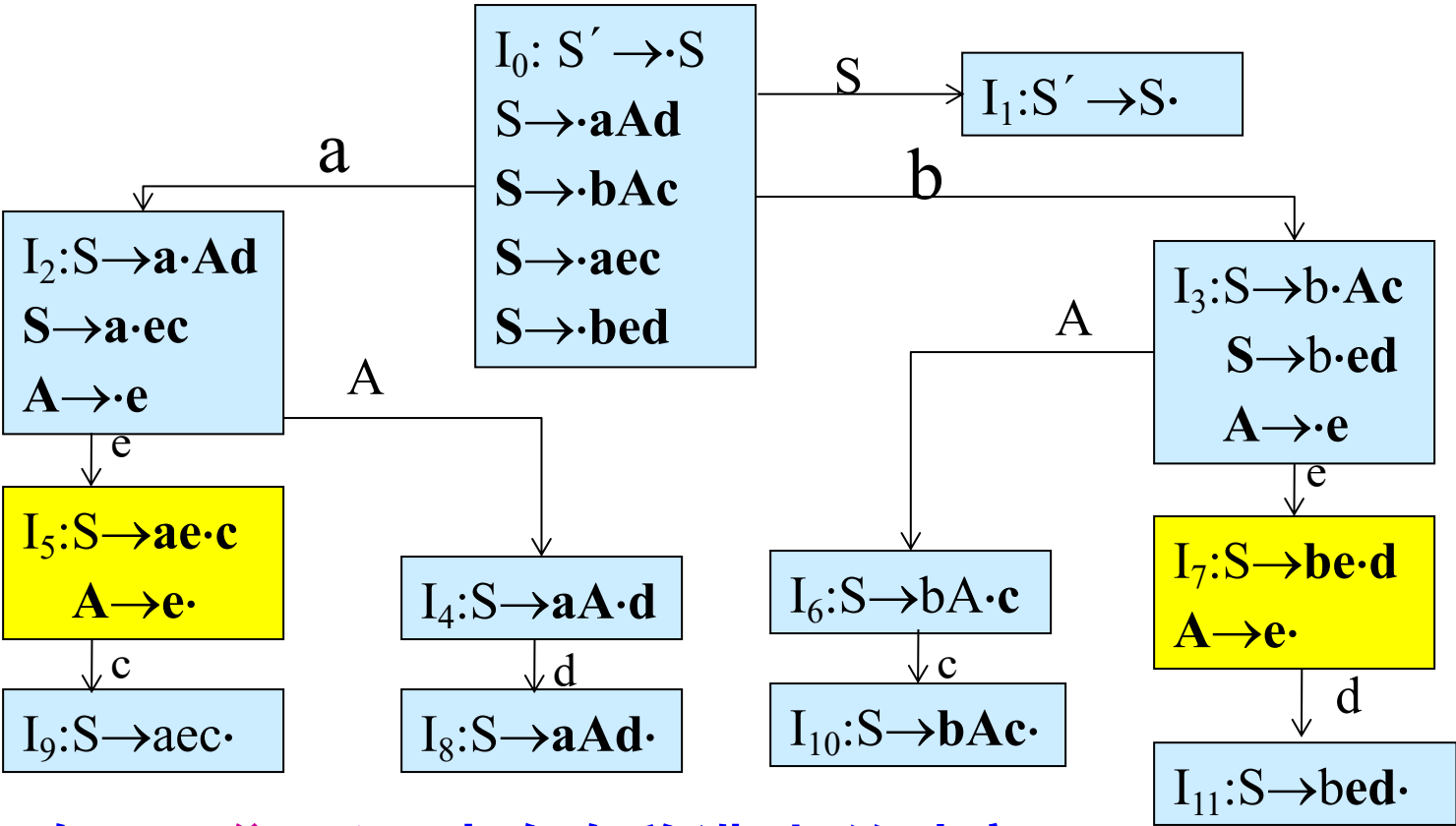
在状态2，面对输入符号=时，存在移进规约冲突，因此，不是SLR(1)文法

不能用SLR(1)分析法进行语法分析的示例

例如文法G'为：

- (0) $S' \rightarrow S$
- (1) $S \rightarrow aAd$
- (2) $S \rightarrow bAc$
- (3) $S \rightarrow aec$
- (4) $S \rightarrow bed$
- (5) $A \rightarrow e$

用 $S' \rightarrow \cdot S$ 作为初态集的项目，
然后用闭包函数和转换函数构造识别文法G'的识别活前缀的DFA如图所示。



在项目集I₅和I₇中存在移进-归约冲突;
 $Follow(A)=\{c,d\}$,与 $\{c\}$, $\{d\}$ 相交,
不能使用SLR(1)分析方法。

SLR(1)冲突问题分析

文法 G' 为:

(0) $S' \rightarrow S$

(1) $S \rightarrow aAd$

(2) $S \rightarrow bAc$

(3) $S \rightarrow aec$

(4) $S \rightarrow bed$

(5) $A \rightarrow e$

1. 产生式右部相似导致的移进-归约冲突

观察产生式: (1) $S \rightarrow aAd$ (3) $S \rightarrow aec$ (5) $A \rightarrow e$

当分析输入串以 ae 开头时, 分析器在处理 a 后, 遇到 e , 有两种可能:

① 归约 $A \rightarrow e$, 然后继续匹配 d , 对应产生式 (1);

② 直接匹配 (3) 产生式的 aec

这导致在某些状态下, 分析器既可以选择归约 $A \rightarrow e$, 也可以选择移进 e 继续匹配 (3), 产生移进-归约冲突。

类似情况也发生在以 b 开头的产生式 (2) 和 (4)。

2. FOLLOW 集无法区分归约时机

SLR(1) 使用 $FOLLOW(A)$ 来判断何时归约 $A \rightarrow e$ 。

由于 A 在产生式 (1) 和 (2) 中后面分别跟着 d 和 c , $FOLLOW(A)$ 包含 $\{c, d\}$ 。

但在分析过程中, 遇到 e 时, 无法通过 $FOLLOW$ 集区分是否应归约或继续移进, 导致冲突。

SLR(1)冲突问题分析得到的结论

以上分析表明，SLR(1)中解决LR(0)冲突的FOLLOW集可能会大于实际即将面临的输入符号集。不妨把即将面临的输入符号集称为超前信息。LR(1)分析法就是利用超前信息解决以上冲突的。

解决方法：通过分裂状态，使LR分析器的每个状态能确切知道句柄 α 后紧跟哪些终结符时才能把 α 归约成A。

LR分析法概念回顾

LR分析法是一种**自底向上的语法分析方法**，适用于广泛的上下文无关文法。

“L”表示从左到右扫描输入，“R”表示最右推导的逆序（Rightmost derivation）。

LR分析法通过**构造状态自动机和分析表**，实现对输入串的有效分析。

LR(1)中的“1”

LR(1)中的“1”表示向前看符号（lookahead）的个数为1。

向前看符号用于判断归约时机，解决了SLR(1)和LR(0)中因上下文信息不足导致的冲突。

一个文法如果可以构造出无冲突的LR(1)分析表，则称该文法为LR(1)文法。

LR(1)文法是LR文法的一个重要子类，能够处理比SLR(1)和LR(0)更复杂的文法。

LR(1)分析方法步骤

1. 增广文法

在原文法基础上添加新的开始符号 $S' \rightarrow S$ ，确保分析的唯一入口。

2. 构造LR(1)项目

一个LR(1)项目形式为： $[A \rightarrow \alpha \bullet \beta, a]$

① $A \rightarrow \alpha \beta$ 是文法产生式

② “ $\bullet$ ”表示当前分析位置，点号左边的符号已经分析完，右边待分析

③ a 是向前看符号（lookahead），表示在归约时，只有当输入符号是 a 时，才允许归约

LR(1)分析方法步骤(续)

3. 计算项目集族

- ① **Closure操作**: 对项目集进行闭包, 加入所有可能的项目, 结合向前看符号的First集计算。
- ② **Goto操作**: 对项目集和文法符号计算转移, 构造状态自动机。

LR(1)分析方法步骤(续)

4. 构造分析表

- ① 根据项目集族构造ACTION和GOTO表。
- ② ACTION表包含移进、归约、接受和错误动作。
- ③ GOTO表用于非终结符的状态转移。

5. 语法分析过程

- ① 利用分析表和状态栈，对输入串进行移进和归约操作。
- ② 当遇到接受状态，分析成功；遇到错误，分析失败。

LR(1)项目集的构造步骤

LR(1)项目集族是所有LR(1)项目的集合，构造过程主要包括：

- ① 初始项目集的构造
- ② 项目集的闭包（Closure）计算
- ③ 项目集的转移（Goto）计算
- ④ 迭代计算直到不再产生新项目集

1. 初始项目集

增广文法：添加新的开始符号 $S' \rightarrow S$

初始项目为： $I_0 = \text{Closure}(\{[S' \rightarrow \bullet S, \$]\})$ 其中，\$表示输入结束符。

例如增广文法 G' ：

(0) $S' \rightarrow S$

(1) $S \rightarrow aAd$

(2) $S \rightarrow bAc$

(3) $S \rightarrow aec$

(4) $S \rightarrow bed$

(5) $A \rightarrow e$

构造初始项目： $I_0 = \text{Closure}(\{[S' \rightarrow \bullet S, \$]\})$

2. 计算闭包 Closure(I)

给定一个LR(1)项目集 I，计算其闭包 Closure(I) 的过程：

1. 初始化 $\text{Closure}(I) = I$
2. 对 Closure(I) 中每个项目 $[A \rightarrow \alpha \bullet B \beta, a]$ ，其中点号后是非终结符 B，执行以下操作：
 - ① 计算 $\text{First}(\beta a)$ ，即串 β 后接向前看符号 a 的First集。
 - ② 对 B的每个产生式 $B \rightarrow \gamma$ ，将项目 $[B \rightarrow \bullet \gamma, b]$ 加入 Closure(I)，其中 $b \in \text{First}(\beta a)$ 。
3. 重复上述过程，直到没有新的项目加入。

计算闭包 Closure(I)的示例

增广文法G'：

(0) $S' \rightarrow S$ (1) $S \rightarrow aAd$ (2) $S \rightarrow bAc$ (3) $S \rightarrow aec$ (4) $S \rightarrow bed$ (5) $A \rightarrow e$

初始项目： $I_0: [S' \rightarrow \bullet S, \$]$

闭包操作：点后是非终结符 S，加入 S 的所有产生式，向前看符号为 $\{\$ \}$ 。

对 Closure(I) 中每个项目 $[A \rightarrow \alpha \bullet B \beta, a]$ ，其中点号后是非终结符 B，执行以下操作：

- ① 计算 $\text{First}(\beta a)$ ，即串 β 后接向前看符号 a 的 First 集。
- ② 对 B 的每个产生式 $B \rightarrow \gamma$ ，将项目 $[B \rightarrow \bullet \gamma, b]$ 加入 Closure(I)，其中 $b \in \text{First}(\beta a)$ 。

$I_0: [S' \rightarrow \bullet S, \$]$
 $[S \rightarrow \bullet aAd, \$]$
 $[S \rightarrow \bullet bAc, \$]$
 $[S \rightarrow \bullet aec, \$]$
 $[S \rightarrow \bullet bed, \$]$

3. 计算转移 $\text{Goto}(I, X)$

1. 对项目集 I 和文法符号 X （终结符或非终结符），定义：

$$\text{Goto}(I, X) = \text{Closure}(\{[A \rightarrow \alpha X \cdot \beta, a] \mid [A \rightarrow \alpha \cdot X \beta, a] \in I\})$$

2. 即将所有点号后为 X 的项目，点号向右移动一位后，计算闭包。

计算转移 Goto(I, X)的项目集示例1

I_0 : $[S' \rightarrow \bullet S, \$]$
 $[S \rightarrow \bullet aAd, \$]$
 $[S \rightarrow \bullet bAc, \$]$
 $[S \rightarrow \bullet aec, \$]$
 $[S \rightarrow \bullet bed, \$]$

对 I_0 的项目，根据点后符号进行GOTO。

GOTO(I_0 , S) 项目 $[S' \rightarrow \bullet S, \$]$ 点后是 S，移点后：

I_1 : $[S' \rightarrow S\bullet, \$]$

4. 构造完整项目集族

- ① 从初始项目集 I_0 开始, 计算所有可能的 Goto 转移, 得到新的项目集。
- ② 对新项目集重复 Closure 和 Goto 计算。
- ③ 直到不再产生新的项目集, 构造完成。

计算GOTO(I_0 , a)

I_0 : $[S' \rightarrow \bullet S, \$]$
 $[S \rightarrow \bullet aAd, \$]$
 $[S \rightarrow \bullet bAc, \$]$
 $[S \rightarrow \bullet aec, \$]$
 $[S \rightarrow \bullet bed, \$]$

涉及项目: $[S \rightarrow \bullet aAd, \$]$
 $[S \rightarrow \bullet aec, \$]$

点后是 a, 移点后: $[S \rightarrow a\bullet Ad, \$]$
 $[S \rightarrow a\bullet ec, \$]$

闭包操作:

- ① 对 $[S \rightarrow a\bullet Ad, \$]$ 点后是非终结符 A, 向前看符号为 d (因为 d 是 A 后的第一个终结符)
- ② 对 $[S \rightarrow a\bullet ec, \{\$ \}]$ 点后是终结符 e, 不扩展。

I_2 : $[S \rightarrow a\bullet Ad, \$]$
 $[S \rightarrow a\bullet ec, \$]$
 $[A \rightarrow \bullet e, d]$

计算GOTO(I_0 , b)

I_0 : $[S' \rightarrow \bullet S, \$]$
 $[S \rightarrow \bullet aAd, \$]$
 $[S \rightarrow \bullet bAc, \$]$
 $[S \rightarrow \bullet aec, \$]$
 $[S \rightarrow \bullet bed, \$]$

涉及项目: $[S \rightarrow \bullet bAc, \$]$
 $[S \rightarrow \bullet bed, \$]$

点后是 b, 移点后: $[S \rightarrow b\bullet Ac, \$]$
 $[S \rightarrow b\bullet ed, \$]$

闭包操作:

- ① 对 $[S \rightarrow b\bullet Ac, \$]$ 点后是非终结符 A, 向前看符号为 c (因为 c 是 A 后的第一个终结符)
- ② 对 $[S \rightarrow b\bullet ed, \{\$ \}]$ 点后是终结符 e, 不扩展。

I_3 : $[S \rightarrow b\bullet Ac, \$]$
 $[S \rightarrow b\bullet ed, \$]$
 $[A \rightarrow \bullet e, c]$

计算GOTO(I_2 , A)

I_2 : $[S \rightarrow a \cdot A d, \$]$

$[S \rightarrow a \cdot e c, \$]$

$[A \rightarrow \cdot e, d]$

涉及项目: $[S \rightarrow a \cdot A d, \$]$

点后是 A, 移点后: $[S \rightarrow a A \cdot d, \$]$

点后是终结符 d, 不扩展。

I_4 : $[S \rightarrow a A \cdot d, \$]$

计算GOTO(I_2 , e)

I_2 : $[S \rightarrow a \cdot A d, \$]$
 $[S \rightarrow a \cdot e c, \$]$
 $[A \rightarrow \cdot e, d]$

涉及项目: $[S \rightarrow a \cdot e c, \$]$
 $[A \rightarrow \cdot e, d]$

点后是e, 移点后: $[S \rightarrow a e \cdot c, \$]$
 $[A \rightarrow e \cdot, d]$

$[A \rightarrow e \cdot, d]$ 是规约项目, 不扩展。

I_5 : $[S \rightarrow a e \cdot c, \$]$
 $[A \rightarrow e \cdot, d]$

计算GOTO(I_3 , A)

I_3 : $[S \rightarrow b \bullet A c, \$]$
 $[S \rightarrow b \bullet e d, \$]$
 $[A \rightarrow \bullet e, c]$

涉及项目: $[S \rightarrow b \bullet A c, \$]$

点后是 A, 移点后: $[S \rightarrow b A \bullet c, \$]$

点后是终结符 c, 不扩展。

I_6 : $[S \rightarrow b A \bullet c, \$]$

计算GOTO(I_3 , e)

I_3 : $[S \rightarrow b \bullet Ac, \$]$
 $[S \rightarrow b \bullet ed, \$]$
 $[A \rightarrow \bullet e, c]$

涉及项目: $[S \rightarrow b \bullet ed, \$]$
 $[A \rightarrow \bullet e, c]$

点后是 A, 移点后: $[S \rightarrow be \bullet d, \$]$
 $[A \rightarrow e \bullet, c]$

点后是终结符 d, 不扩展。

I_7 : $[S \rightarrow be \bullet d, \$]$
 $[A \rightarrow e \bullet, c]$

计算GOTO(I_4 , d)

I_4 : $[S \rightarrow a A \bullet d, \$]$

涉及项目: $[S \rightarrow a A \bullet d, \$]$

点后是 d, 移点后: $[S \rightarrow a A d \bullet, \$]$

点在最后后, 是规约项目

I_8 : $[S \rightarrow a A d \bullet, \$]$

计算GOTO(I_5 , c)

I_5 : $[S \rightarrow a e \bullet c, \$]$
 $[A \rightarrow e \bullet, d]$

涉及项目: $[S \rightarrow a e \bullet c, \$]$

点后是 c, 移点后: $[S \rightarrow a e c \bullet, \$]$

I_9 : $[S \rightarrow a e c \bullet, \$]$

计算GOTO(I_6 , c)

I_6 : $[S \rightarrow b A \bullet c, \$]$

涉及项目: $[S \rightarrow b A \bullet c, \$]$

点后是 c, 移点后: $[S \rightarrow b A c \bullet, \$]$

I_{10} : $[S \rightarrow b A c \bullet, \$]$

计算GOTO(I_7 , d)

I_7 : $[S \rightarrow be \bullet d, \$]$
 $[A \rightarrow e \bullet, c]$

涉及项目: $[S \rightarrow be \bullet d, \$]$

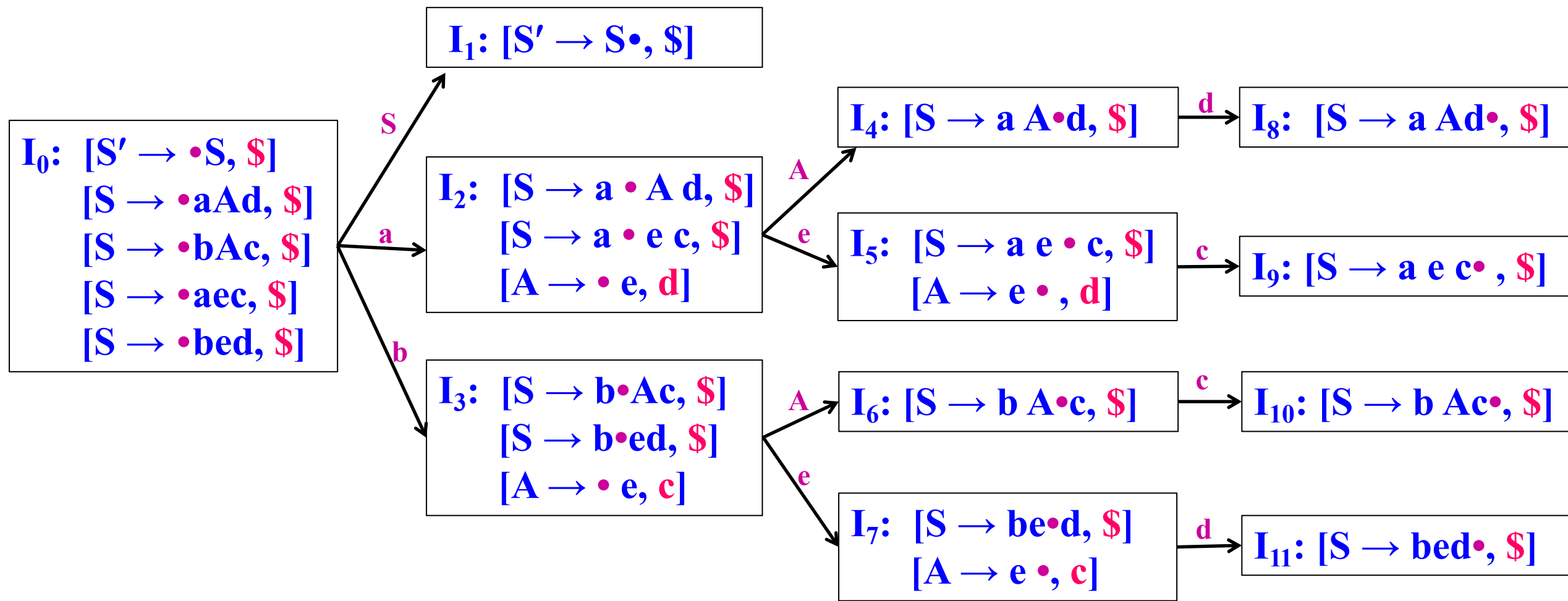
点后是 d, 移点后: $[S \rightarrow bed \bullet, \$]$

I_{11} : $[S \rightarrow bed \bullet, \$]$

LR(1)项目集构成的有限状态自动机

增广文法G':

(0) $S' \rightarrow S$ (1) $S \rightarrow aAd$ (2) $S \rightarrow bAc$ (3) $S \rightarrow aec$ (4) $S \rightarrow bed$ (5) $A \rightarrow e$



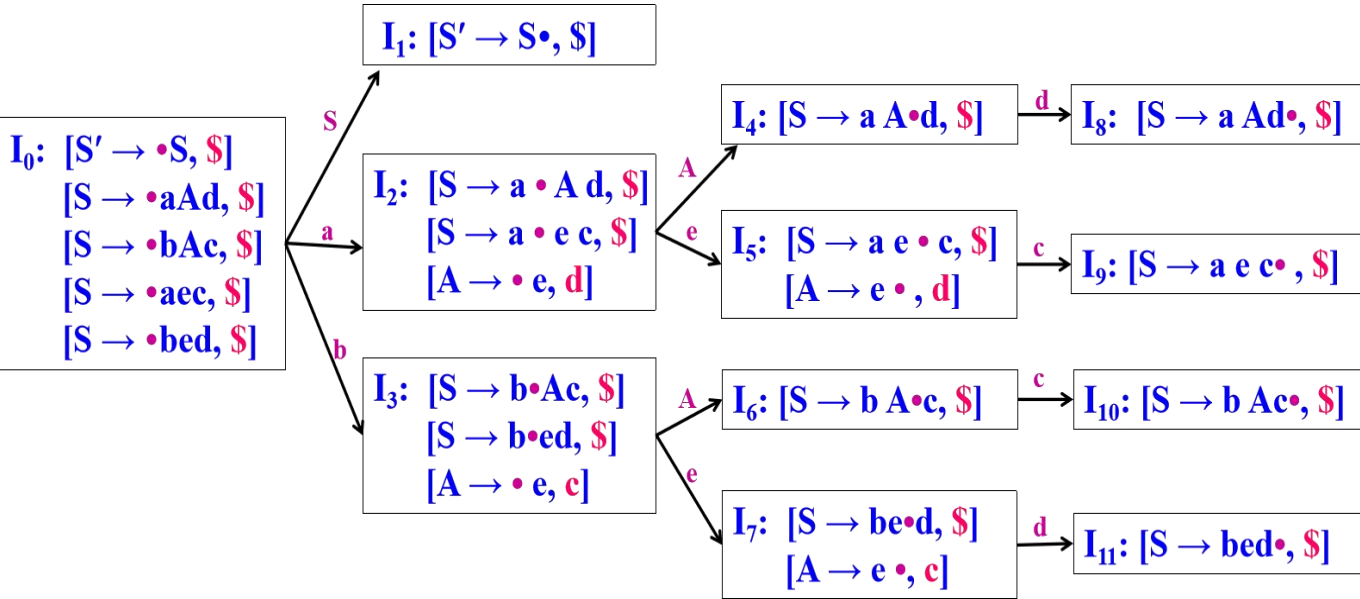
LR(1)分析表构造如下:

1. 若项目 $[A \rightarrow \alpha \cdot X \beta, b] \in I_k$, 且 $GOTO(I_k, X) = I_j$,
 - ① $X = a \in V_T$, 则置 $ACTION[k, a] = S_j$, 意为把 (j, a) 移进栈;
 - ② $X \in V_N$, 则置 $GOTO(k, X) = j$
2. 若项目 $[A \rightarrow \alpha \cdot, a] \in I_k$, 且 $A \neq S'$, 对于 a , 则置 $ACTION[k, a] = r_j$, 意为用第 j 个产生式 $A \rightarrow \alpha$ 进行归约
3. 若 $[S' \rightarrow S \cdot, \$] \in I_k$, 则置 $ACTION[k, \$] = \text{"accept"}$
4. 凡不能用以上规则填入的入口, 置 "error"

构造LR(1)的文法分析表

增广文法G'：

- (0) $S' \rightarrow S$ (1) $S \rightarrow aAd$
- (2) $S \rightarrow bAc$ (3) $S \rightarrow aec$
- (4) $S \rightarrow bed$ (5) $A \rightarrow e$



状态	ACTION						GOTO	
	a	b	c	d	e	\$	S	A
0	s ₂	s ₃					1	
1						accept		
2					s ₅			4
3					s ₇			6
4				s ₈			8	
5			s ₉	r ₅				
6			s ₁₀					
7			r ₅	s ₁₁				
8						r ₁		
9						r ₃		
10						r ₂		
11						r ₄		

计算LR(1)项目集的示例2

■ 有增广文法 $G(S')$ 第1步: $I_0 = \{ [S' \rightarrow \cdot S, \$] \}$

(0) $S' \rightarrow S$

第2步: 计算 $\text{closure}(S) = [S \rightarrow \cdot CC, \$]$,

(1) $S \rightarrow CC$

则 $I_0 = \{ [S \rightarrow \cdot S, \$], [S \rightarrow \cdot CC, \$] \}$

(2) $C \rightarrow cC$

第3步: 由于 $[S \rightarrow \cdot CC, \$]$, 第2个C的 $\text{first}(C) = \{c, d\}$,

(3) $C \rightarrow d$

计算 $\text{closure}(C) = \{ [C \rightarrow \cdot cC, c/d], [C \rightarrow \cdot d, c/d] \}$,

则 $I_0 = \{ [S \rightarrow \cdot S, \$], [S \rightarrow \cdot CC, \$], [C \rightarrow \cdot cC, c/d], [C \rightarrow \cdot d, c/d] \}$

即得到项集:

$S' \rightarrow \cdot S, \$$	I_0
$S \rightarrow \cdot CC, \$$	
$C \rightarrow \cdot cC, c/d$	
$C \rightarrow \cdot d, c/d$	

LR(1)的GOTO(I,X)函数

令I是一个项集，X是一个文法符号，函数GOTO(I, X)定义为：

$$\text{GOTO}(I, X) = \text{CLOSURE}(J)$$

其中： $J = \{\text{任何形如}[A \rightarrow \alpha X \cdot \beta, a] \text{的项目} \mid [A \rightarrow \alpha \cdot X \beta, a] \in I\}$

LR(1)有限自动机的示例

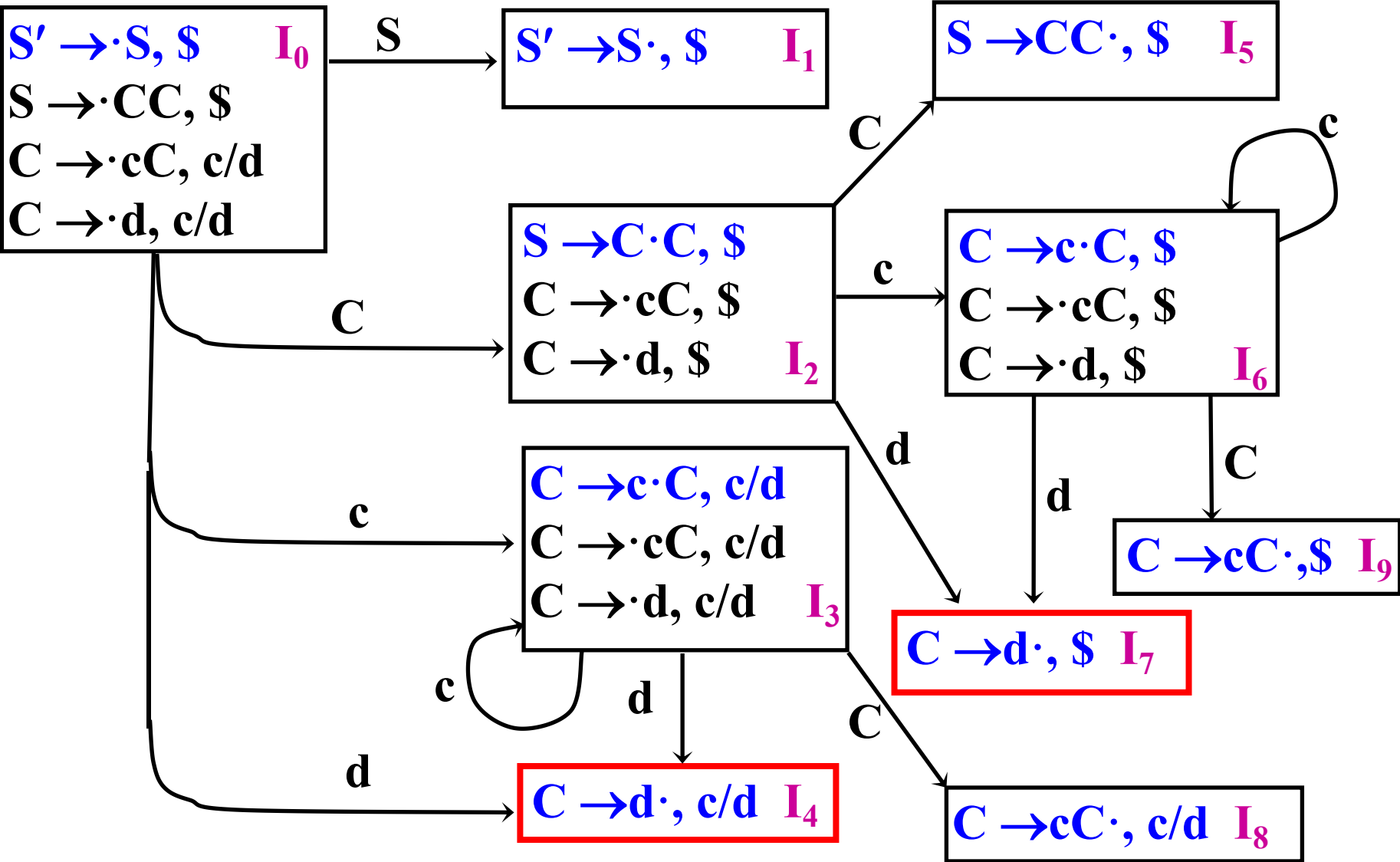
■ 有增广文法G(S')

(0) $S' \rightarrow S$

(1) $S \rightarrow CC$

(2) $C \rightarrow cC$

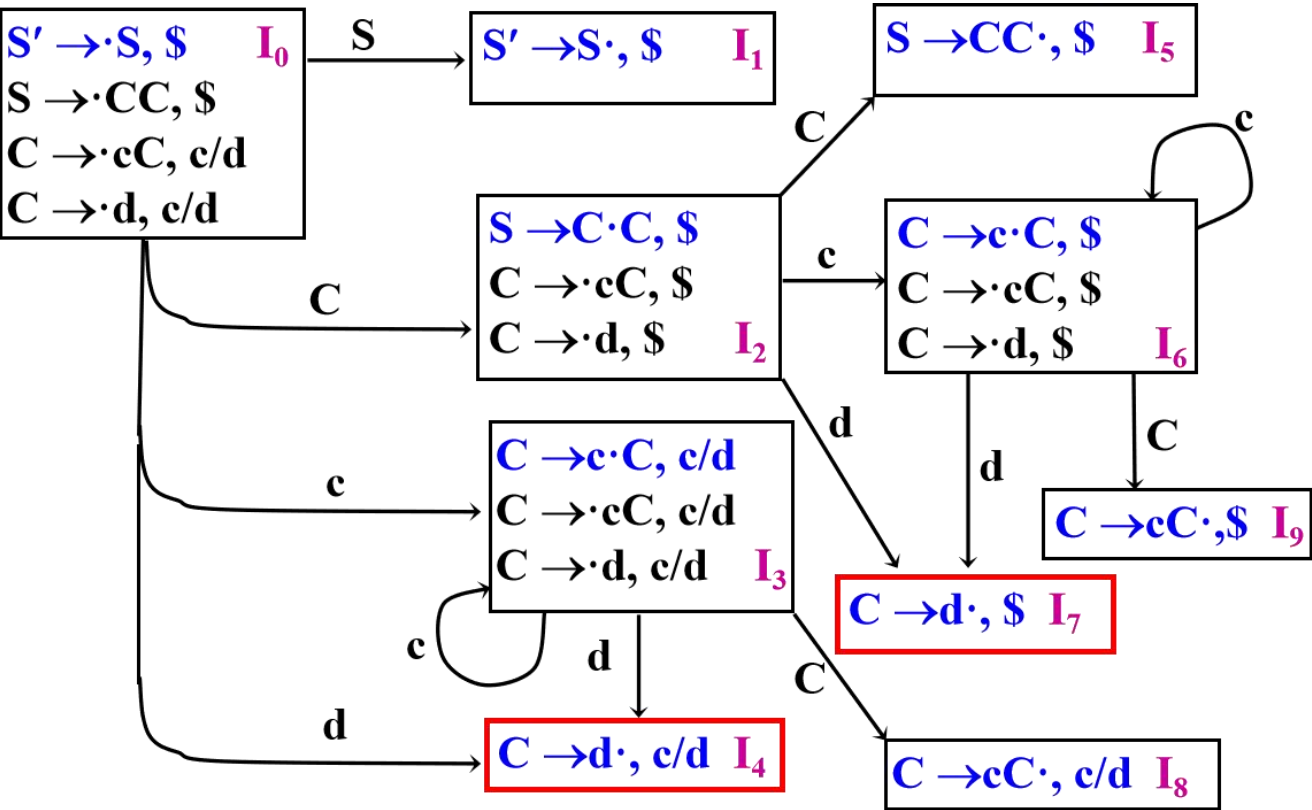
(3) $C \rightarrow d$



LR(1)分析表构造如下:

1. 若项目 $[A \rightarrow \alpha \cdot X \beta, b] \in I_k$, 且 $GOTO(I_k, X) = I_j$,
 - ① $X = a \in V_T$, 则置 $ACTION[k, a] = S_j$, 意为把 (j, a) 移进栈;
 - ② $X \in V_N$, 则置 $GOTO(k, X) = j$
2. 若项目 $[A \rightarrow \alpha \cdot, a] \in I_k$, 且 $A \neq S'$, 对于 a , 则置 $ACTION[k, a] = r_j$, 意为用第 j 个产生式 $A \rightarrow \alpha$ 进行归约
3. 若 $[S' \rightarrow S \cdot, \$] \in I_k$, 则置 $ACTION[k, \$] = \text{"accept"}$
4. 凡不能用以上规则填入的入口, 置 "error"

LR(1)分析表示例



状态	ACTION			GOTO	
	c	d	\$	S	C
0	s3	s4		1	2
1			accept		
2	s6	s7			5
3	s3	s4			8
4	r3	r3			
5			r1		
6	s6	s7			9
7			r3		
8	r2	r2			
9			r2		

注意：ACTION[k,a]= S_j,意为把(j,a)移进栈，状态变化到项目集I_j

ACTION[k,a]= r_j,意为用第j个产生式A→α进行归约；

GOTO中的 J={任何形如[A→αX·β, a]的项目集编号 | [A→α·Xβ, a]∈I}

例: 按上表对ccdcd进行分析

步骤	状态	符号	输入串	动作
0	0	\$	ccdcd\$	移入c
1	03	\$c	cdcd\$	移入c
2	033	\$cc	dcd\$	移入d
3	0334	\$ccd	cd\$	C→d归约
4	0338	\$ccC	cd\$	C→cC归约
5	038	\$cC	cd\$	C→cC归约
6	02	\$C	cd\$	移进c
7	026	\$Cc	d\$	移进c
8	0267	\$Ccd	\$	C→d归约
9	0269	\$CcC	\$	C→cC归约
10	025	\$CC	\$	S→CC归约
11	01	\$S	\$	接受

状态	ACTION			GOTO	
	c	d	\$	S	C
0	s3	s4		1	2
1			acc		
2	s6	s7			5
3	s3	s4			8
4	r3	r3			
5			r1		
6	s6	s7			9
7			r3		
8	r2	r2			
9			r2		

- (0) S'→S
- (1) S→CC
- (2) C→cC
- (3) C→d

LR(1)分析法的不足

- 随着文法的复杂度增加，LR(1)项集族的大小也会增长，导致分析表变得庞大，占用更多的内存空间。这可能会影响分析的效率和速度。
- 相对于其他自底向上的分析方法，如SLR(1)，LR(1)分析法的性能可能不如其它方法高效。LR(1)项集族的构建和分析表的生成可能需要更多的时间和资源。
- 由上例中可以看到，有些状态集除了搜索符不同外是两两相同的
- 解决办法：合并同心集，构造LALR分析表

合并同心集算法

- ① 构造文法G的LR(1)项集族 $C = \{I_0, I_1, \dots, I_n\}$
- ② 把所有的同心集合并在一起，合并后的新族为 $C = \{J_0, J_1, \dots, J_m\}$ 。含有项目 $[S' \rightarrow S, \$]$ 的 J_k 为分析表的初态
- ③ 从新族C构造ACTION表
- ④ 构造GOTO表: 若 $goto(I_k, X) = J_j$, 则 $goto[k, X] = j$;
- ⑤ 分析表中凡不能用③④填入信息的空白格均填上“出错标志”

从C构造ACTION子表

- ① 若项目 $[A \rightarrow \alpha \cdot a \beta, b] \in J_k$ 且 $GO(J_k, a) = J_j$, a 为终结符, 则 $ACTION[k, a]$ 为 s_j ;
- ② 若项目 $[A \rightarrow \alpha \cdot, a]$ 属于 J_k , 则置 $ACTION[k, a]$ 为 “用产生式 $A \rightarrow \alpha$ 归约”, 简记为 r_j ; 其中, 假定 $A \rightarrow \alpha$ 为文法 G' 的第 j 个产生式;
- ③ 若项目 $[S' \rightarrow S \cdot, \$]$ 属于 J_k , 则置 $ACTION[k, \$]$ 为 “接受”, 简记为 $accept$;

LALR分析表示例

■ 文法 $G(S')$

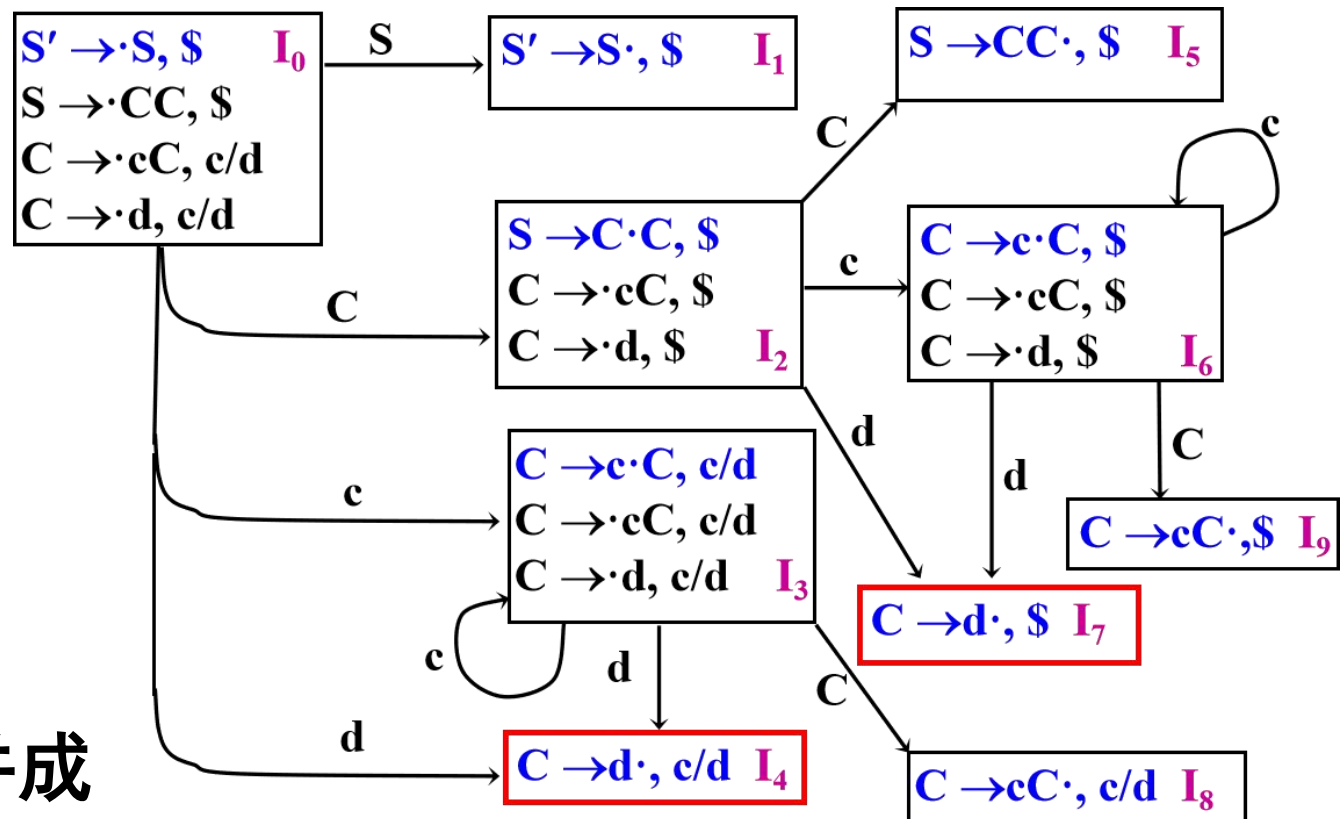
(0) $S' \rightarrow S$

(1) S → CC

(2) $C \rightarrow \mathbf{c}C$

(3) $C \rightarrow d$

①把状态3, 6、4, 7和8, 9分别合并成



I₃₆: [C->c·C,c/d/\$]

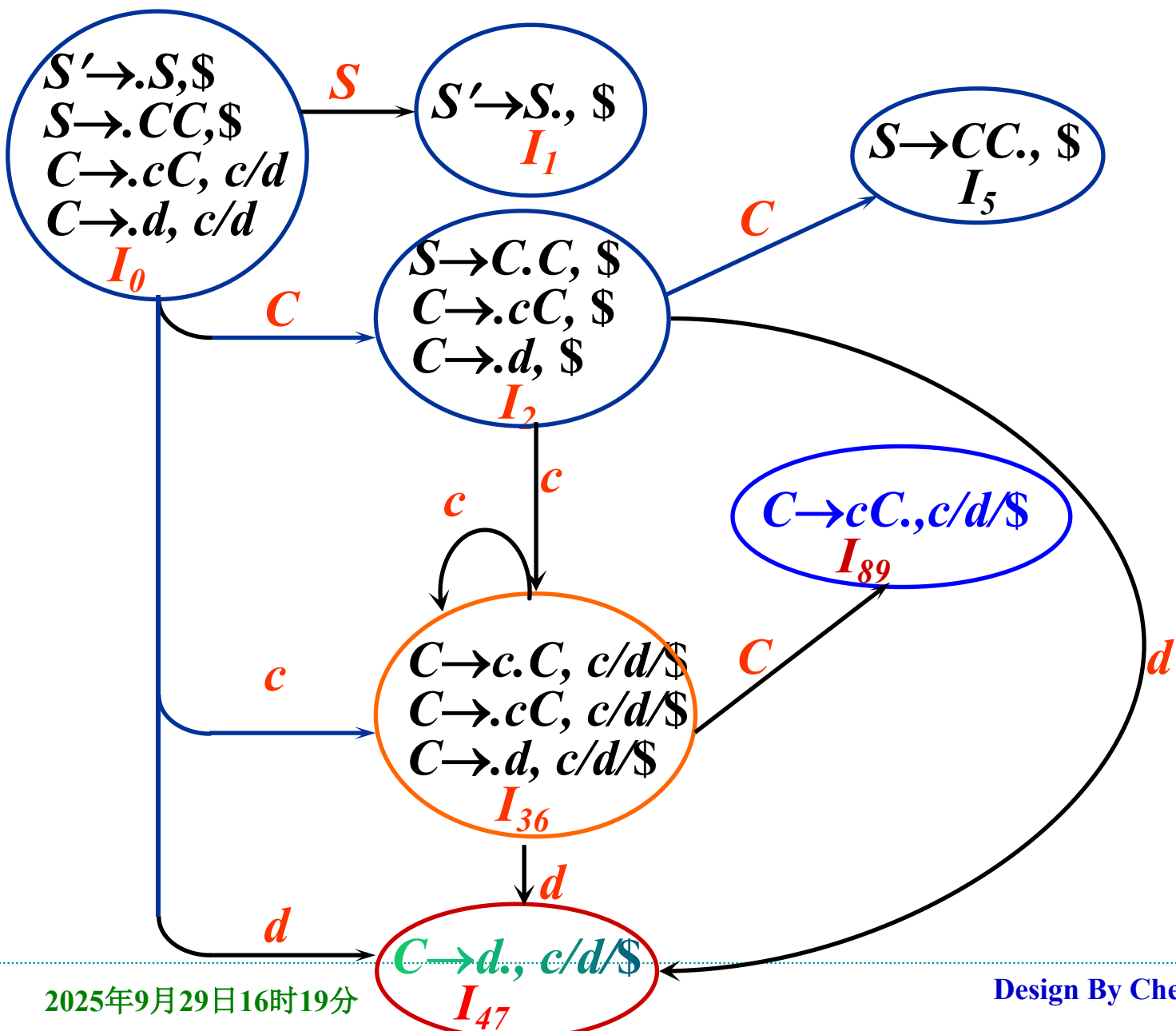
[C->·cC,c/d/\$]

[C->·d,c/d/\$]

I₄₇: [C->d·,c/d/\$]

I₈₉: [C->cC·,c/d/\$]

② 由合并后的集族所构成的LALR分析表



状态	ACTION(动作)			GOTO(转换)	
	c	d	\$	S	C
0	s36	s47		1	2
1			acc		
2	s36	s47			5
36	s36	r47			89
47	r3	r3	r3		
5			r1		
89	r2	r2	r2		

几种至下而上分析法在归约时的差异

- ① LR(0) 分析考虑所有终结符
- ② SLR(1) 分析考虑FOLLOW 集
- ③ LR(1) 分析仅考虑 LR(1)项目中的搜索符
- ④ LALR分析考虑的是LR(1)项目合并之后的搜索符

几种至下而上分析法在构造分析表上的差异



本章课后作业

1. 给定文法 $G: S \rightarrow aSb \mid \varepsilon$ 对于输入串 $aaabbb$, 使用LR(0)分析法进行语法分析。
2. 给定文法 $G: E \rightarrow E+E \mid E * E \mid (E) \mid id$ 对于输入串 $id+id*id$, 使用LL(1)分析法进行语法分析。
3. 给定文法 $G: S \rightarrow aSb \mid T, T \rightarrow cTd \mid \varepsilon$ 对于输入串 $acccdbd$, 使用SLR(1)分析法进行语法分析。
4. 给定文法 $G: E \rightarrow E+T \mid T, T \rightarrow T * F \mid F, F \rightarrow (E) \mid id$ 对于输入串 $id * (id+id)$, 使用LR(1)分析法进行语法分析。
5. 给定文法 $G: S \rightarrow aS \mid bS \mid c$ 对于输入串 $ababcc$, 使用LALR(1)分析法进行语法分析。

本章课后作业

6. 给定文法 $G: S \rightarrow aSb \mid cSc \mid \varepsilon$ 对于输入串 $acbcc$, 使用LL(1)分析法进行语法分析。
7. 给定文法 $G: E \rightarrow E+E \mid E * E \mid (E) \mid id$ 对于输入串 $id * (id + id)$, 使用SLR(1)分析法进行语法分析。
8. 给定文法 $G: S \rightarrow aS \mid bS \mid c$ 对于输入串 $ababcc$, 使用LR(1)分析法进行语法分析。
9. 给定文法 $G: E \rightarrow E+T \mid T, T \rightarrow T * F \mid F, F \rightarrow (E) \mid id$ 对于输入串 $id * (id + id)$, 使用LALR(1)分析法进行语法分析。
10. 给定文法 $G: S \rightarrow aSb \mid T, T \rightarrow cTd \mid \varepsilon$ 对于输入串 $acccdbd$, 使用LL(1)分析法进行语法分析。